

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



GoodAccess CLI klient pro Linux

BAKALÁŘSKÁ PRÁCE

Vypracoval: Valdemar Pospíšil

Vedoucí práce: Ing. Jiří Hromádka

Studijní program: Aplikovaná informatika

Studijní obor:

ÚSTÍ NAD LABEM 2026

Namísto žlutých stránek vložte digitálně podepsané zadání kvalifikační práce poskytnuté vedoucím katedry.

Zadání musí zaujímat právě dvě strany.

Zadání je nutno vložit jako PDF pomocí některého nástroje, který umožňuje editaci dokumentů (se zachováním elektronického podpisu).

V Linuxe lze například použít příkaz `pdftk`.

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 18. března 2026

Podpis:

Děkuji vedoucímu práce **Ing. Jiřímu Hromádkovi**
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Abstrakt:

V současné době nabízí GoodAccess VPN řešení především prostřednictvím grafického uživatelského rozhraní, což nevyhovuje všem uživatelským scénářům. Zejména v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů preferují administrátoři a pokročilí uživatelé řešení příkazové řádky.

Hlavním cílem této práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích s využitím WireGuard protokolu. Práce se zabývá návrhem architektury, bezpečným ukládáním dat a implementací komunikace se systémovou službou pomocí IPC.

Klíčová slova: VPN, GoodAccess, CLI, Linux, Go, .NET, WireGuard, IPC, Clean Architecture

GOODACCESS CLI CLIENT FOR LINUX

Abstract:

Currently, GoodAccess VPN offers solutions primarily through a graphical user interface, which does not suit all user scenarios. Especially in server environments, automated systems, and DevOps workflows, administrators and advanced users prefer command-line solutions.

The main goal of this work is to design and implement a functional prototype of a CLI application for managing GoodAccess VPN connections on Linux distributions using the WireGuard protocol. The work deals with the design of the architecture, secure data storage, and implementation of communication with the system service using IPC.

Keywords: VPN, GoodAccess, CLI, Go, .NET, WireGuard

Obsah

Úvod	15
1. Teoretická východiska	17
1.1. Rozhraní příkazové řádky	17
1.2. Architektura VPN a principy Zero Trust	19
1.3. Architektura operačního systému Linux	22
1.4. Meziprocetová komunikace (IPC) v Linuxu	24
1.5. Technologie a nástroje	25
1.6. Zajištění kvality a testování softwaru	29
1.7. Distribuce softwaru a balíčkovací systémy	30
1.8. Agilní metodiky a Extreme Programming	32
2. Analýza a specifikace požadavků	35
2.1. Identifikace zúčastněných stran	35
2.2. Případy užití	35
2.3. Funkční požadavky	36
2.4. Nefunkční požadavky	37
2.5. Analýza architektury	38
3. Návrh řešení	41
3.1. Komponenty systému	41
3.2. Návrh komunikace IPC	42
3.3. Návrh uživatelského rozhraní (CLI)	44
3.4. Správa dat a bezpečnostní model	47
3.5. Návrh distribuce a aktualizací	49
4. Implementace	51
4.1. Struktura projektu a vývojové prostředí	51
4.2. Implementace systémové služby (.NET)	53
4.3. Implementace klientské aplikace (Go)	56
4.4. Pokročilé funkce a technické výzvy	59
5. Testování a validace	63
5.1. Metodika testování	63
5.2. Jednotkové testy a mockování	63

5.3. Akceptační testování	66
5.4. Systémové a integrační testování v prostředí Linux	66
Závěr	69
A. Externí přílohy	73
B. Další přílohy	75

Seznam použitých zkratek

API Application Programming Interface

APT Advanced Package Tool

BDD Behavior-Driven Development

CI Continuous Integration

CLI Command Line Interface

DAC Discretionary Access Control

E2E End-to-End

FHS Filesystem Hierarchy Standard

GUI Graphical User Interface

IPC Inter-Process Communication

JSON JavaScript Object Notation

OIDC OpenID Connect

PID Process Identifier

POSIX Portable Operating System Interface

QA Quality Assurance

SDLC Software Development Life Cycle

SDP Software Defined Perimeter

SSH Secure Shell

SSO Single Sign-On

TDD Test-Driven Development

TUI Text User Interface

UDS Unix Domain Sockets

UID User Identifier

VPN Virtual Private Network

XP Extreme Programming

ZTNA Zero Trust Network Access

Úvod

V posledních letech došlo k zásadnímu posunu v organizaci práce a správě síťové infrastruktury. S masivním přechodem na práci na dálku a distribuované týmy se tradiční modely zabezpečení sítě ukazují jako nedostatečné. Organizace stále častěji adoptují architektury typu Zero Trust Network Access (ZTNA), kde je bezpečné VPN připojení klíčovým prvkem nejen pro koncové uživatele, ale i pro serverovou infrastrukturu.

Platforma GoodAccess v současné době nabízí řešení primárně prostřednictvím grafického uživatelského rozhraní (GUI). Toto řešení je vhodné pro běžné uživatele na osobních počítačích, avšak naráží na limity v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů.

Motivace

V serverovém prostředí Linux, které je dominantní platformou pro cloudovou infrastrukturu, často není grafické rozhraní dostupné (tzv. headless servery). Administrátoři a DevOps inženýři vyžadují nástroje, které lze ovládat z příkazové řádky, snadno skriptovat a integrovat do CI/CD pipelin. Absence nativního CLI (Command Line Interface) klienta pro Linux představuje pro GoodAccess značné omezení v možnosti nasazení v těchto scénářích.

Cíle práce

Hlavním cílem této bakalářské práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích. Důraz je kladen na vytvoření "lehkého" a nativního uživatelského rozhraní, které efektivně komunikuje se systémovou službou a využívá stávající podnikovou logiku pro správu VPN tunelů. Práce má ambici ukázat kompletní inženýrský proces vývoje softwaru od analýzy požadavků až po distribuci finálních balíčků.

Dílčí cíle práce zahrnují:

- Analýzu a návrh architektury založené na principu oddělení logiky od prezentace (tzv. "stupid frontend" přístup).
- Implementaci meziprocesní komunikace (IPC) mezi uživatelským rozhraním v jazyce Go a systémovým modulem v .NET prostřednictvím Unix Domain Sockets.

- Vytvoření modulu v .NET pro bezpečné řízení VPN tunelů (OpenVPN, WireGuard) s využitím existujících komponent platformy GoodAccess.
- Návrh a realizaci automatizovaného testování a balíčkování aplikace pro distribuce Debian a Fedora.

Struktura práce

Práce je rozdělena do několika logických celků. Kapitola 1 se věnuje teoretickým východiskům, popisuje architekturu Linuxu, použité VPN protokoly a metody meziprocesové komunikace. Kapitola 2 definuje požadavky na systém a analyzuje možné architektonické přístupy. Následující kapitoly se poté věnují samotnému návrhu, implementaci a testování výsledného řešení.

1. Teoretická východiska

Tato kapitola analyzuje klíčové technologie, architektonické principy a bezpečnostní standardy nezbytné pro návrh moderních systémových nástrojů v prostředí OS Linux. Text postupuje od obecné problematiky příkazové řádky přes síťové protokoly až po specifika implementačních platforem .NET a Go.

1.1. Rozhraní příkazové řádky

Rozhraní příkazové řádky (CLI – Command Line Interface) představuje jeden z nejstarších, avšak stále nejvýkonnějších způsobů interakce uživatele s počítačem. Navzdory masivnímu rozšíření grafických uživatelských rozhraní (GUI) v 90. letech 20. století zůstává CLI nepostradatelným nástrojem pro systémové administrátory, vývojáře a pokročilé uživatele, zejména v prostředí operačních systémů typu Unix a Linux.

Role CLI v moderní správě systémů

Efektivita a automatizace

Hlavním důvodem preference příkazové řádky v profesionálním prostředí je efektivita, rychlost a snadná automatizace. Zatímco grafické rozhraní je navrženo pro intuitivní objevování funkcí pomocí vizuálních prvků, CLI se zaměřuje na precizní a opakovatelné provádění úkolů. Administrátor může pomocí jediného příkazu nebo krátkého skriptu provést operaci na stovkách serverů současně, což je v rámci GUI prakticky neproveditelné [1].

Vzdálená správa a cloud

Dalším kritickým faktorem je vzdálená správa. Protokol SSH (Secure Shell), který je de facto standardem pro správu linuxových serverů, je primárně textově orientovaný. Přenos textových příkazů a jejich výstupů vyžaduje minimální síťovou propustnost, což umožňuje spolehlivou správu systémů i přes nestabilní nebo pomalé připojení [2]. V prostředí cloudu a kontejnerizace (např. Docker, Kubernetes), kde systémy často nemají žádné grafické výstupní zařízení (headless režim), je CLI jedinou možností interakce.

Principy návrhu a standardy CLI aplikací

Kvalitní CLI aplikace se řídí souborem ustálených principů, které zajišťují jejich předvídatelnost a vzájemnou spolupráci. Eric S. Raymond ve své knize *The Art of Unix Programming* definuje tzv. „Unixovou filosofii“, jejímž základem je tvorba malých, jednoúčelových nástrojů, které spolu efektivně komunikují [1]. Tento přístup umožňuje skládat komplexní operace z jednoduchých komponent pomocí mechanismu rour (pipes).

Unixová filozofie

Základním stavebním kamenem interakce v CLI jsou standardní datové proudy, které umožňují řetězení příkazů do složitých celků [1]:

- **Standardní vstup (stdin):** Zdroj dat pro aplikaci, typicky klávesnice nebo výstup jiného programu.
- **Standardní výstup (stdout):** Primární kanál pro výstup dat, typicky terminál.
- **Standardní chybový výstup (stderr):** Oddělený kanál pro chybová hlášení a diagnostiku, což umožňuje uživateli přeměrovat data do souboru, zatímco chyby stále vidí na obrazovce.

Standardní datové proudy

Důležitou roli hraje také syntaxe příkazů. Moderní nástroje se snaží dodržovat standardy jako POSIX nebo GNU konvence pro argumenty a přepínače (flags). Krátké přepínače (např. `-v`) jsou určeny pro rychlé psaní, zatímco dlouhé varianty (např. `--verbose`) zvyšují čitelnost skriptů a dokumentace [1].

Syntaxe a standardy

CLI versus TUI

V rámci terminálu se můžeme setkat také s textovým uživatelským rozhraním (TUI – Text User Interface). Na rozdíl od klasického CLI, které vypisuje řádky textu v sekvenčním pořadí, TUI využívá celou plochu terminálu k vykreslování vizuálních prvků, jako jsou okna, menu, tabulky nebo interaktivní grafy, často s využitím knihoven jako `ncurses` nebo moderních frameworků v jazyce Go (např. `Bubble Tea` [3]).

Hlavní rozdíl spočívá v komplexitě a způsobu interakce. CLI je ideální pro jednorázové příkazy a automatizaci v nekonečném proudu (batch processing). TUI je naproti tomu vhodnější pro interaktivní nástroje, kde uživatel potřebuje neustálý přehled o stavu systému nebo kde je navigace v datech pomocí klávesových zkratk efektivnější než vypisování parametrů (např. textové editory, správci souborů nebo dashboardy pro monitoring).

1.2. Architektura VPN a principy Zero Trust

Tradiční pojetí zabezpečení sítě, založené na ochraně perimetru, prochází v posledních letech zásadní transformací. Tato sekce popisuje evoluci od klasických VPN tunelů k moderní architektuře Zero Trust Network Access (ZTNA). Tento vývoj je v současnosti silně ovlivněn nejen technologickým pokrokem, ale i legislativními rámci, jako je evropská směrnice NIS2, která klade zvýšený důraz na odolnost kritické infrastruktury a doporučuje principy Zero Trust jako součást moderní strategie kybernetické bezpečnosti [4].

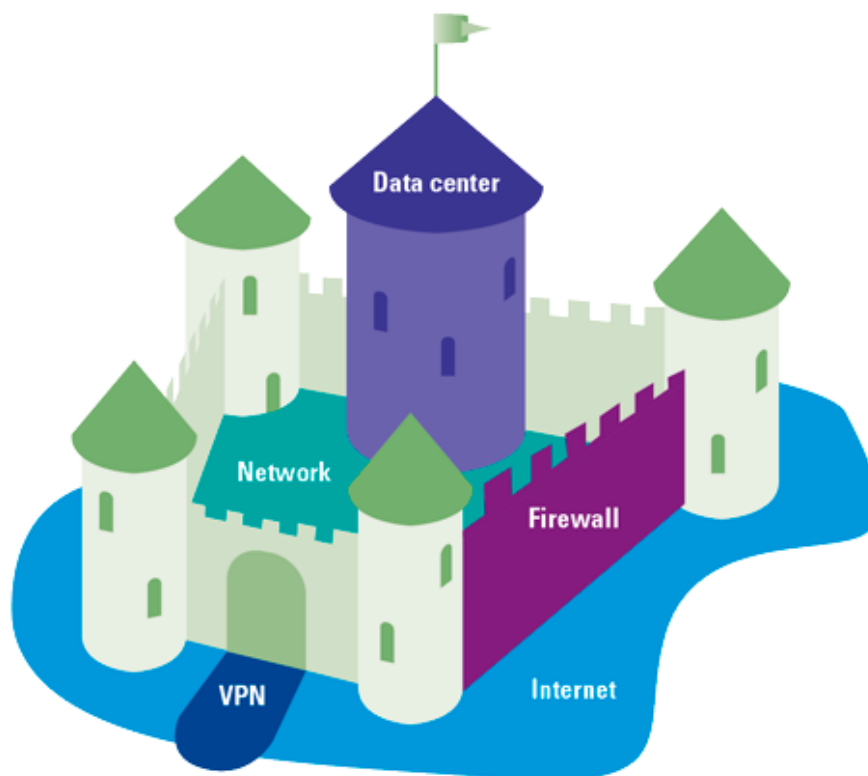
Tradiční VPN a perimetrová bezpečnost

Tunelování a enkapsulace

Virtuální privátní síť (VPN) je technologie, která umožňuje vytvořit bezpečné, šifrované spojení (tunel) mezi dvěma body v rámci nezabezpečené sítě, jako je internet. Princip tunelování spočívá v zapouzdření (enkapsulaci) síťových paketů do jiného protokolu, který zajišťuje integritu a důvěrnost přenášených dat [5].

Model hrad a příkop

Historicky se organizace spoléhaly na model „hrad a příkop“ (Castle-and-Moat). V tomto modelu je vnitřní síť považována za důvěryhodnou zónu, která je od vnějšího světa oddělena firewallem (příkopem) [6]. VPN v tomto kontextu slouží jako „padací most“, který umožňuje vzdáleným uživatelům vstoupit do vnitřní sítě. Jakmile však uživatel získá přístup, má často široký dosah k mnoha zdrojům v rámci sítě. Tento přístup trpí kritickou slabinou: pokud útočník kompromituje jediné zařízení uvnitř perimetru, může se v síti volně pohybovat (laterální pohyb) a napadat další systémy, protože vnitřní síť mu implicitně důvěřuje [7].



Obrázek 1.1.: Vizualizace modelu Castle-and-Moat [6]

Koncept Zero Trust

Filozofie Zero Trust (nulová důvěra) model „hrad a příkop“ opouští a vychází z principu „nikdy nedůvěřuj, vždy prověřuj“ (Never trust, always verify). Samotný koncept není nový; termín „Zero Trust“ zpopularizoval již v roce 2010 John Kindervag (v té době analytik společnosti Forrester Research), který prosazoval myšlenku, že bezpečnost musí být nedílnou součástí architektury sítě, nikoliv pouze jejím vnějším obalem [4]. V konceptu Zero Trust není důvěra nikdy udělena automaticky na základě fyzického umístění uživatele nebo jeho IP adresy. Každý pokus o přístup k jakémukoliv zdroji musí být individuálně autentizován, autorizován a šifrován [4].

Tabulka 1.1.: Srovnání modelů Castle-and-Moat a Zero Trust [8]

Kritérium	Castle-and-Moat	Zero Trust
Filozofie	„Důvěřuj, ale prověřuj“	„Nikdy nedůvěřuj, vždy prověřuj“
Perimetr	Statický (firewall)	Dynamický (identita uživatele)
Úroveň přístupu	Široký přístup k síti	Granulární (Least Privilege)
Ověřování	Jednorázové (při vstupu)	Kontinuální (při každém požadavku)
Laterální pohyb	Snadný po průniku	Výrazně omezen segmentací
Škálovatelnost	Omezená hardwarem	Vysoká (cloud-native)
Viditelnost	Omezená (logy na bráně)	Detailní (každá relace)

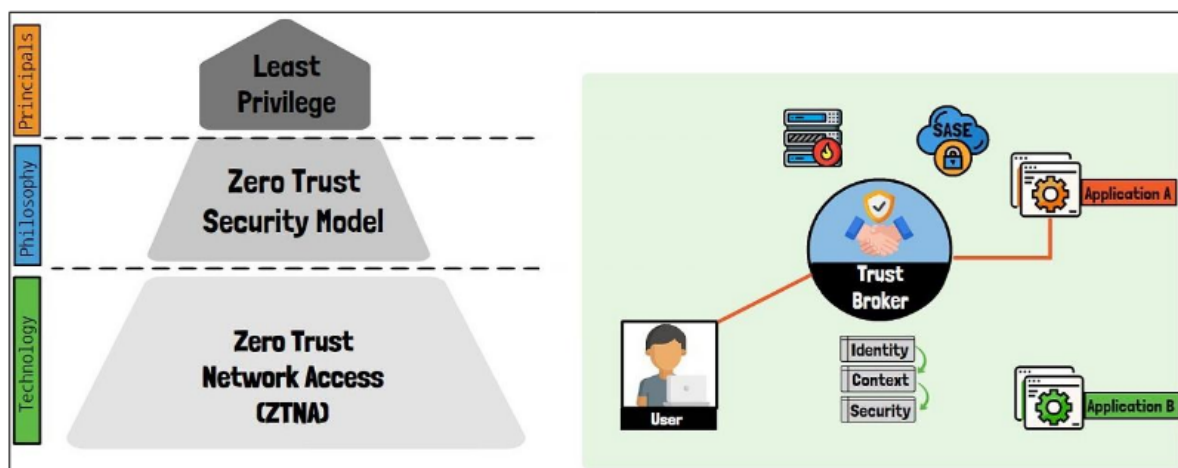
Zero Trust Network Access (ZTNA) a GoodAccess

Zero Trust Network Access (ZTNA) představuje praktickou technologickou implementaci architektury Zero Trust. V architektuře ZTNA je kontrola přístupu striktně vynucována pro každou relaci.

Klíčovým prvkem ZTNA je Software Defined Perimeter (SDP). SDP vytváří dynamickou, virtuální hranici kolem aplikací a služeb, čímž je činí „neviditelnými“ pro veřejný internet i pro neautorizované uživatele v lokální síti. GoodAccess implementuje tyto principy formou cloudové platformy, která nahrazuje tradiční VPN brány [9]. Místo statických IP adres využívá identitu uživatele (často propojenou s SSO/OIDC) a kontext zařízení (tzv. Device Posture) k dynamickému přidělování přístupových práv k jednotlivým systémům [9]. Tím se radikálně snižuje plocha možného útoku, protože uživatel vidí pouze ty zdroje, ke kterým má explicitní oprávnění.

V rámci ZTNA se rozlišují dva hlavní přístupy k segmentaci:

- **Aplikace (Mikrosegmentace):** Rozdělení infrastruktury na úrovni jednotlivých aplikací a služeb. Uživatel získává přístup pouze k těm systémům, které nezbytně potřebuje ke své práci (např. vývojář má přístup k repositářům, ale nikoliv k účetnímu systému). Tento přístup efektivně brání laterálnímu pohybu útočníka v síti [4].
- **Síť (Makrosegmentace):** Tradičnější přístup rozdělující síť na větší logické celky. Moderní ZTNA řešení však umožňují realizovat makrosegmentaci softwarově bez nutnosti složité konfigurace fyzických síťových prvků [4].



Obrázek 1.2.: Architektura Zero Trust Network Access [10]

Protokol WireGuard

WireGuard je moderní VPN protokol navržený s důrazem na jednoduchost, rychlost a moderní kryptografii [5]. S rozsahem přibližně 4 000 řádků kódu je WireGuard velmi snadno

auditovatelný. Díky své implementaci přímo v jádře Linuxu dosahuje vynikající propustnosti a velmi nízké latence. Z pohledu správy systému se WireGuard chová jako standardní síťové rozhraní, což výrazně zjednodušuje konfiguraci směrování a celkovou integraci do systémových nástrojů. Pro dynamické enterprise prostředí, které GoodAccess obsluhuje, představuje WireGuard ideální volbu především díky mimořádně rychlému navazování spojení (handshake) a vysoké odolnosti vůči změnám síťového prostředí (roaming).

1.3. Architektura operačního systému Linux

Tato sekce analyzuje klíčové mechanismy operačního systému Linux, které jsou nezbytné pro běh systémových služeb a správu síťových rozhraní.

Struktura systému: User Space a Kernel Space

Linux dělí paměťový prostor na dvě privilegovaně odlišné části. Toto rozdělení je kritické pro stabilitu a bezpečnost systému.

Prostor jádra a uživatelský prostor

- **Kernel Space (Prostor jádra):** Zde běží jádro operačního systému, ovladače zařízení a kritické síťové moduly, včetně implementace protokolu WireGuard. Kód v tomto prostoru má plný přístup k hardwaru.
- **User Space (Uživatelský prostor):** Zde běží běžné aplikace, včetně uživatelských rozhraní a backendových služeb.

Toto oddělení, často označované jako *privilege separation*, zajišťuje, že chybný nebo škodlivý kód v uživatelské aplikaci nemůže přímo ohrozit integritu celého systému. Brian Ward ve své knize *How Linux Works* zdůrazňuje, že zatímco uživatelský prostor je místem, kde probíhá většina „reálné akce“ aplikací, jádro slouží jako arbitr a správce všech hardwarových prostředků [2].

Systemová volání a přepnutí kontextu

Aplikace v uživatelském prostoru nemohou přímo přistupovat k hardwaru. Musí využívat tzv. *systemová volání* (syscalls) pro komunikaci s jádrem. Přejít mezi těmito prostory vyžaduje tzv. *přepnutí kontextu* (context switch), což je operace s nenulovou režií. V kontextu VPN řešení je toto kritické při zpracování síťového provozu — například protokoly běžící v uživatelském prostoru musí každý paket kopírovat mezi prostory přes virtuální síťové rozhraní (TUN/TAP), což může omezovat propustnost při vysokém zatížení. Naopak implementace přímo v prostoru jádra tuto režií minimalizují a umožňují efektivnější zpracování datových toků [5].

Správa procesů a životní cyklus služeb

Proces je v Linuxu definován jako instance běžícího programu v paměti. Každý proces má svůj unikátní identifikátor (PID) a je spravován jádrem, které mu přiděluje procesorový čas a paměťové prostředky.

Procesy a daemony

Vytváření nových procesů probíhá pomocí dvojice systémových volání `fork()` a `exec()`. Volání `fork()` vytvoří identickou kopii rodičovského procesu, zatímco `exec()` nahradí kód běžícího procesu novým programem. Tento mechanismus je základem pro spouštění všech aplikací v systému [2].

Specifickou kategorií procesů jsou *daemony*. Jedná se o procesy, které běží na pozadí, nejsou přímo spojeny s žádným terminálem a typicky čekají na určité události nebo poskytují systémové služby. V moderní architektuře systémových nástrojů bývá logika implementována právě jako daemon, který udržuje stav (např. VPN spojení) nezávisle na tom, zda je uživatelské rozhraní zrovna spuštěno. Tento přístup zajišťuje persistenci spojení a umožňuje bezpečné provádění privilegovaných operací (např. manipulaci se síťovými rozhraními) pod identitou uživatele s dostatečnými právy, zatímco samotné rozhraní může běžet v kontextu běžného uživatele.

Správa služeb a systémová inicializace (systemd)

Moderní linuxové distribuce využívají pro správu systému a služeb inicializační systém `systemd`. Ten v posledním desetiletí nahradil starší standardy jako `sysvinit` a přinesl zásadní změny v tom, jak jsou procesy na pozadí spouštěny, monitorovány a hierarchicky organizovány.

systemd a správa služeb

Zatímco starší systémy spouštěly služby sekvenčně pomocí shell skriptů, což prodlužovalo start systému, `systemd` využívá agresivní paralelizaci a spouštění služeb na základě požadavků (on-demand), například skrze socketovou aktivaci. Brian Ward vysvětluje, že `systemd` není pouhým inicializačním procesem s PID 1, ale komplexním ekosystémem pro správu uživatelského prostoru, který integruje logování (`journald`), správu zařízení (`udev`) i konfiguraci sítě [2].

Z pohledu vývoje systémových nástrojů jsou klíčové tzv. *Unit files* (jednotky), konkrétně `.service` soubory. Tyto deklarativní konfigurační soubory definují parametry spuštění služby, její závislosti na síťové konektivitě a bezpečnostní kontext. Díky integraci s mechanismem `cgroups` (Control Groups) má `systemd` absolutní kontrolu nad všemi procesy patřícími k dané

službě, což umožňuje jejich spolehlivé ukončení nebo automatický restart v případě havárie. V architektuře postavené na službách hraje **systemd** roli garanta běhu, zajišťuje spuštění po startu systému a poskytuje standardizované rozhraní pro monitoring stavu skrze nástroj **systemctl**.

Konfigurace síťového subsystému (Netlink)

Tradiční unixové nástroje ('ifconfig') byly v Linuxu nahrazeny modernějším balíkem 'iproute2'. Na pozadí těchto nástrojů stojí rozhraní **Netlink**.

Netlink slouží jako komunikační sběrnice mezi jádrem a uživatelským prostorem [2]. Na rozdíl od 'ioctl' volání, Netlink používá standardní mechanismus socketů pro předávání zpráv o konfiguraci sítě (např. přiřazení IP adresy, změna routovací tabulky).

1.4. Meziprocesová komunikace (IPC) v Linuxu

Jelikož je aplikace rozdělena na dvě části (privilegovaná služba a neprivilegovaný klient), je nutné zajistit efektivní přenos dat mezi nimi. V prostředí Linuxu je standardem pro lokální komunikaci využití **Unix Domain Sockets (UDS)**.

Unix Domain Sockets

Na rozdíl od TCP/IP socketů, které využívají síťová rozhraní a IP adresy, UDS využívají souborový systém. Socket je reprezentován speciálním souborem na disku [2]. Hlavní výhody pro systémové služby jsou:

1. **Výkon:** Data se nekopírují přes síťový zásobník, ale zůstávají v paměti OS.
2. **Bezpečnost a řízení přístupu:** Přístup k socketu lze omezit standardními linuxovými oprávněními souborů. To umožňuje, aby se k chráněné službě mohl připojit pouze uživatel s příslušným oprávněním.

Správa oprávnění a souborový systém

Linux využívá standard **FHS (Filesystem Hierarchy Standard)**, který definuje, kam mají aplikace ukládat svá data. Pro software třetích stran je vyhrazen adresář **/opt**, zatímco konfigurace a logy patří do **/etc** a **/var/log**.

Model oprávnění DAC a bezpečnostní kontext

Linux využívá model Discretionary Access Control (DAC), kde každý soubor má vlastníka, skupinu a sadu práv (**rwX**) [2]. Pro systémové služby jako VPN klient je však tento model často nedostačující, protože operace se síťovým rozhraním vyžadují práva uživatele **root**.

V rámci bezpečného návrhu běží kritické komponenty pod identitou **root** (typicky jako systemd služba), zatímco frontend komunikuje přes Unix Domain Socket s omezeným přístupem. Moderní Linux dále rozšiřuje tento model o tzv. *Capabilities* (schopnosti), které umožňují rozdělit absolutní moc uživatele **root** na menší celky (např. **CAP_NET_ADMIN** pro správu sítě), čímž implementují princip nejmenších privilegií [2].

1.5. Technologie a nástroje

V rámci teoretického základu pro vývoj moderních systémových nástrojů je nezbytné analyzovat vlastnosti platforem a technologií, které jsou pro tento typ úloh optimalizovány. Tato sekce se zaměřuje na platformu .NET 8, jazyk Go a transportní formáty dat, přičemž zkoumá jejich vnitřní architekturu a bezpečnostní mechanismy.

Platforma .NET 8

Platforma .NET 8 (dříve .NET Core) představuje moderní, multiplatformní vývojový ekosystém navržený pro vysoký výkon a škálovatelnost. Jedním z klíčových aspektů této platformy je její schopnost nativního běhu v prostředí OS Linux, což z ní činí vhodný nástroj pro vývoj serverových aplikací a systémových služeb [11].

Generic Host a životní cyklus služeb

Základním architektonickým prvkem pro běh aplikací na pozadí je tzv. **Generic Host**. Tento vzor poskytuje standardizované rozhraní pro správu životního cyklu aplikace, konfiguraci, logování a delegování úloh skrze mechanismus *Dependency Injection* (DI) [12]. Generic Host sjednocuje způsob, jakým jsou spravovány různé typy služeb, od webových serverů až po jednoduché úlohy běžící na pozadí. V kontextu systémových služeb umožňuje precizní řízení procesu ukončení (*graceful shutdown*), kdy aplikace při přijetí signálu k ukončení (např. SIGTERM) korektně uvolní prostředky a ukončí běžící operace, čímž předchází nekonzistenci dat.

Integrace se systémem systemd

Pro hlubší integraci s linuxovým inicializačním systémem **systemd** poskytuje platforma specializovaný modul `Microsoft.Extensions.Hosting.Systemd` [12]. Tato komponenta umožňuje .NET aplikaci nativně komunikovat s inicializačním systémem **systemd**. Díky této integraci může služba odesílat stavová hlášení (např. o úspěšném spuštění) přímo do **systemd** a využívat pokročilé funkce správy služeb, jako je automatické obnovení po havárii nebo logování skrze systémový žurnál (**journald**).

ASP.NET Core Data Protection

Kritickou součástí moderních systémů je ochrana citlivých dat, kterou v ekosystému .NET zajišťuje stack **ASP.NET Core Data Protection** [13]. Tento mechanismus je navržen k šifrování dat „v klidu“ (encryption at rest) a k bezpečné správě kryptografických klíčů. Architektura stacku je založena na hierarchickém systému klíčů, kde je každý datový prvek šifrován unikátním klíčem odvozeným z hlavního klíče (master key).

Bezpečnostní stack řeší několik klíčových problémů automatizovaně:

- **Izolace:** Klíče jsou vázány na konkrétní aplikaci, což zabraňuje dešifrování dat jiným procesem i na stejném systému.
- **Rotace:** Systém automaticky generuje nové klíče a zneplatňuje staré, čímž omezuje dopad případného úniku klíče.
- **Perzistence:** Na systému Linux stack podporuje ukládání klíčů ve formátu XML (Extensible Markup Language) do souborového systému, přičemž je možné využít šifrování klíčů pomocí certifikátů nebo systémových úložišť.

Díky této abstrakci může vývojář pracovat s jednoduchým rozhraním `IDataProtector`, zatímco platforma transparentně zajišťuje kryptografickou integritu a důvěrnosti uložených dat.

Programovací jazyk Go

Jazyk Go, vyvinutý společností Google, se stal standardem pro systémové programování a cloud-native nástroje. Jeho filozofie je založena na minimalismu, efektivitě a bezpečnosti. Go kombinuje výkon kompilovaných jazyků (jako C++) s produktivitou jazyků s dynamickou typovou kontrolou [14].

Charakteristika jazyka a systémové programování

Hlavní předností Go v oblasti systémových nástrojů je jeho schopnost statické kompilace. Výsledkem procesu sestavení je jediný binární soubor, který v sobě obsahuje všechny nezbytné knihovny (včetně runtime prostředí). To je v prostředí Linuxu zásadní výhoda, neboť uživatel nemusí instalovat žádné závislosti, což radikálně zjednodušuje distribuci softwaru [15]. Go dále disponuje pokročilým modelem souběžnosti založeným na *gorutinách* a kanálech (*channels*), což umožňuje efektivní asynchronní zpracování síťových operací a událostí bez složitosti tradičních vláken.

Framework Cobra

Pro tvorbu rozhraní příkazové řádky (CLI) se v ekosystému Go prosadil framework **Cobra** [16]. Ten definuje standardizovanou strukturu pro definici příkazů, podřízených příkazů a je-

jich parametrů (včetně rozlišení mezi globálními a lokálními přepínači). Cobra je de facto standardem, na kterém staví nástroje jako Docker či Kubernetes. Framework automatizuje generování nápovědy, validaci argumentů a poskytuje rozhraní pro generování manuálových stránek nebo skriptů pro automatické doplňování v shellu, čímž zajišťuje konzistentní uživatelský zážitek.

Architektura Elm a framework Bubble Tea

V oblasti interaktivních textových rozhraní (TUI) představuje špičku framework **Bubble Tea** [3]. Ten je unikátní tím, že do terminálového prostředí přináší principy funkcionálního programování skrze vzor **The Elm Architecture** [17]. Tato architektura definuje tři základní komponenty:

1. **Model:** Datová struktura reprezentující aktuální stav aplikace.
2. **Update:** Čistá funkce, která přijímá zprávu (*Message*) a aktuální model a vrací model nový společně s případnými příkazy (*Commands*).
3. **View:** Funkce, která na základě modelu vygeneruje textovou reprezentaci rozhraní pro terminál.

Zásadním přínosem tohoto přístupu je determinismus a snadná testovatelnost. Veškeré změny stavu probíhají asynchronně skrze zasílání zpráv, což zabraňuje vzniku *race conditions*. Framework Bubble Tea se stará o efektivní překreslování terminálu (tzv. *diffing*), kdy jsou aktualizovány pouze ty části obrazovky, které se skutečně změnily. To umožňuje vytvářet plynulá a reaktivní rozhraní, která uživateli poskytují okamžitou zpětnou vazbu bez problikávání či zpoždění.

Testovací knihovny a nástroje

Pro implementaci automatizovaných testů v různých programovacích jazycích a vrstvách aplikace byly zvoleny následující nástroje:

- **Go testing:** Standardní knihovna jazyka Go poskytuje základní mechanismy pro psaní, spouštění a orchestraci jednotkových testů bez nutnosti instalace externích závislostí [18].
- **Testify:** Tato nadstavba nad standardním Go testováním rozšiřuje jeho možnosti o pokročilé aserce a především o robustní podporu pro mockování (vytváření atrap) závislostí, což je klíčové pro izolované testování komponent [19].
- **xUnit:** Pro ekosystém .NET představuje xUnit moderní a vysoce rozšiřitelný framework pro psaní jednotkových a integračních testů, který klade důraz na čistotu a jednoduchost testovacího kódu [20].

Serializace dat a transportní formáty

Efektivní výměna dat mezi procesy (IPC) vyžaduje volbu vhodného formátu pro serializaci, který balancuje mezi výkonem, čitelností a snadnou údržbou.

Formát JSON

JSON (JavaScript Object Notation) je široce rozšířený, textově orientovaný formát [21]. Jeho hlavní výhodou je transparentnost – přenášená data lze snadno monitorovat a analyzovat běžnými systémovými nástroji. JSON nevyžaduje definici schématu předem, což usnadňuje agilní vývoj a integraci s webovými API. Navzdory textové povaze je parsování moderními knihovnami extrémně rychlé a pro většinu IPC scénářů na jednom hostiteli je režie zanedbatelná.

Protokol gRPC a Protocol Buffers

Naproti tomu **gRPC** je binární framework využívající **Protocol Buffers** jako mechanismus pro definici dat (*Interface Definition Language*) [22]. Je navržen pro maximální propustnost a minimální režii, čehož dosahuje díky binárnímu kódování a generování silně typovaného kódu pro různé programovací jazyky. gRPC vyniká v systémech s vysokým zatížením a v prostředí mikroslužeb, kde redukce velikosti zpráv vede k úspoře síťových prostředků.

Srovnání a volba transportní vrstvy

Srovnání těchto technologií ukazuje na odlišné priority. gRPC dominuje v oblasti čistého výkonu a typové bezpečnosti, vyžaduje však složitější proces sestavení aplikace a ztěžuje manuální inspekci dat [22]. JSON naopak zůstává preferovanou volbou pro systémy, kde je kladen důraz na diagnostiku a jednoduchost implementace.

Tabulka 1.2.: Srovnání formátů JSON a gRPC [22]

Kritérium	JSON	gRPC (Protobuf)
Typ dat	Textový (UTF-8)	Binární
Schéma	Volitelné	Povinné (.proto)
Výkon	Nižší (parsování textu)	Vysoký (binární serializace)
Čitelnost	Výborná pro člověka	Vyžaduje dekodér
Podpora prohlížečů	Nativní	Vyžaduje proxy

V rámci lokální komunikace mezi systémovými nástroji, kde je důležitá možnost nahlédnout do komunikace mezi klientem a službou bez nutnosti binární dešifrace, představuje JSON optimální kompromis mezi technickou efektivitou a vývojovou udržitelností.

1.6. Zajištění kvality a testování softwaru

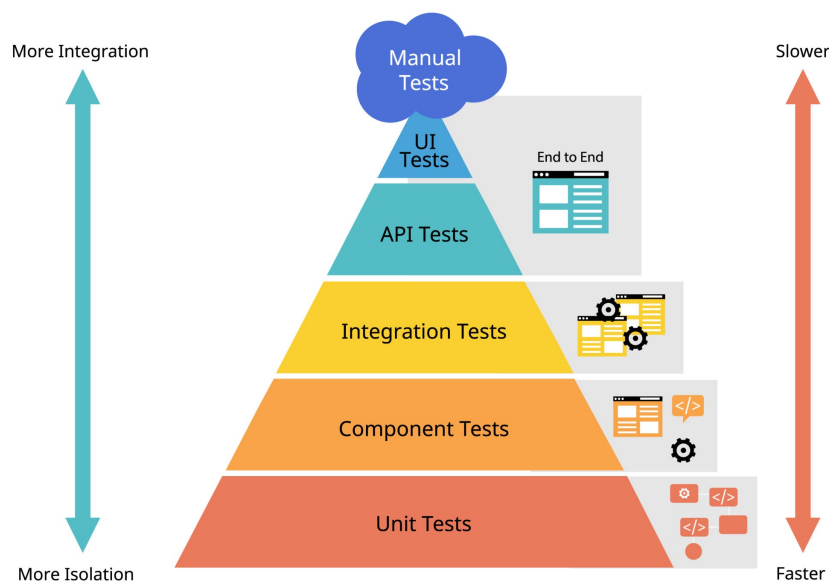
Kvalita softwaru není náhodným výsledkem, ale důsledkem systematického procesu, který doprovází celý životní cyklus vývoje (SDLC – Software Development Life Cycle). V moderním inženýrství, zejména u systémových nástrojů, které manipulují se síťovým nastavením a vyžadují vysoká oprávnění, je verifikace správnosti klíčovým prvkem pro zajištění stability a bezpečnosti systému.

Kvalita softwaru v SDLC

Tradiční pohled na testování jako na oddělenou fázi na konci vývoje je v agilním prostředí nahrazen integrací testů přímo do procesu tvorby kódu. Zajištění kvality (QA – Quality Assurance) zahrnuje nejen detekci chyb, ale především prevenci jejich vzniku skrze statickou analýzu, code review a automatizované testování [23]. Automatizace v tomto procesu umožňuje rychlou regresní analýzu při jakýchkoliv změnách v systému.

Testovací pyramida

Koncept testovací pyramidy, který poprvé popularizoval Mike Cohn v knize *Succeeding with Agile*, definuje ideální distribuci různých typů automatizovaných testů [23]. Cílem je vytvořit robustní základnu rychlých a levných testů, které jsou doplněny menším množstvím komplexnějších a pomalejších scénářů.



Obrázek 1.3.: Model testovací pyramidy podle Mikea Cohna [23]

Pyramida se skládá ze tří hlavních úrovní:

- **Jednotkové testy (Unit Tests):** Tvoří základnu pyramidu. Testují jednotlivé funkce nebo třídy v izolaci od okolního prostředí (např. pomocí mockování). Jsou extrémně rychlé a poskytují vývojáři okamžitou zpětnou vazbu.
- **Integrační testy (Integration Tests):** Zaměřují se na interakci mezi různými moduly nebo externími službami, čímž ověřují správnou komunikaci mezi komponentami systému.
- **End-to-End testy (E2E):** Vrchol pyramidu. Simulují reálné chování uživatele a testují systém jako celek od vstupu v CLI až po vytvoření VPN tunelu v jádře. Jsou nejpomalejší a nejvíce náchylné k chybám v konfiguraci prostředí (tzv. flakiness).

Behavior-Driven Development (BDD) a Gherkin

Vývoj řízený chováním (BDD – Behavior-Driven Development) je metodika, která rozšiřuje principy TDD o zaměření na uživatelský pohled a čitelnost scénářů pro netechnické členy týmu [3]. Specifikace jsou psány v přirozeném jazyce pomocí syntaxe **Gherkin**.

Scénáře v Gherkinu využívají klíčová slova:

- **Given (Za předpokladu):** Definice výchozího stavu systému.
- **When (Když):** Akce, kterou uživatel nebo systém provede.
- **Then (Pak):** Očekávaný výsledek operace.

Tento přístup umožňuje vytvořit tzv. „živou dokumentaci“, která je zároveň spustitelným testem. V rámci moderního inženýrství jsou Gherkin scénáře často využity pro precizní definici požadavků na rozhraní a chování systému, což zajišťuje, že implementace přesně odpovídá navržené specifikaci.

1.7. Distribuce softwaru a balíčkovací systémy

Správa a distribuce softwaru v ekosystému Linuxu představuje komplexní proces, který zajišťuje integritu, bezpečnost a snadnou údržbu systému. Na rozdíl od monolitických operačních systémů, kde je instalace softwaru často v režii jednotlivých aplikací, Linux využívá centralizované balíčkovací systémy, které spravují software jako soubor vzájemně propojených komponent.

Hierarchie distribucí a model Upstream/Downstream

~~Linuxový ekosystém je charakteristický svou fragmentací, která je však organizována do logických struktur. Základem jsou tzv. foundational distributions (např. Debian, Fedora, Arch Linux, Slackware), ze kterých se odvozují stovky dalších variant.~~

Vztah mezi vývojáři a uživateli je definován modelem Upstream/Downstream [24]:

- **Upstream (Horní proud):** Představuje autory původního zdrojového kódu (např. projekt Linux kernel, GNOME, nebo samotný vývojář aplikace). Zde probíhá primární vývoj a opravy chyb.
- **Downstream (Dolní proud):** Představuje linuxové distribuce, které přebírají kód z upstreamu, upravují jej pro své potřeby (např. aplikací specifických patchů), kompilují a balí do binárních balíčků.

Příkladem hierarchie je rodina Debian: Debian (foundational) → Ubuntu (downstream vůči Debianu) → Linux Mint (downstream vůči Ubuntu). Tato hierarchie umožňuje efektivní sdílení oprav a inovací — oprava v upstreamu postupně „proteče“ všemi downstream distribucemi k uživatelům.

Koncepty správy balíčků

Moderní správa balíčků je založena na třech pilířích:

1. **Metadata:** Každý balíček obsahuje podrobné informace o své verzi, architektuře, popisu a především o svých závislostech.
2. **Závislosti (Dependencies):** Schopnost systému automaticky identifikovat a doinstalovat knihovny nebo jiné programy, které balíček vyžaduje ke svému běhu. Tím se předchází tzv. „dependency hell“, kdy uživatel musel ručně dohledávat potřebné komponenty.
3. **Repozitáře:** Centralizovaná úložiště spravovaná distribucí, která slouží jako jediný důvěryhodný zdroj softwaru. To radikálně zvyšuje bezpečnost oproti stahování instalátorů z neznámých webových stránek.

Ekosystém Debian (.deb)

Formát balíčků `.deb` je standardem pro rodinu Debian. Teoretický základ Debian packagingu definuje Debian Policy Manual [25]. Balíček se skládá ze dvou hlavních částí: datového archivu (obsahujícího samotné binární soubory a dokumentaci) a kontrolního archivu.

Klíčovým prvkem je adresář `debian/`, který obsahuje:

- **control:** Definuje metadata jako název, verzi, sekci a pole `Depends` pro runtime závislosti a `Build-Depends` pro nástroje potřebné ke kompilaci.
- **rules:** Prováděcí skript (typicky Makefile), který definuje, jak se má software kompilovat a instalovat do dočasného adresáře před zabalením.
- **Maintainer scripts:** Skripty (`preinst`, `postinst`, `prerm`, `postrm`), které jádro systému spouští před instalací nebo po ní pro konfiguraci systémových služeb.

Ekosystém Red Hat a Fedora (.rpm)

Ecosystem založený na RPM (Red Hat Package Manager) je de facto standardem pro podnikovou sféru (RHEL) a komunitní distribuci Fedora. Standardy pro tuto rodinu definují Fedora Packaging Guidelines [26].

Zatímco Debian využívá sadu souborů v adresáři `debian/`, RPM staví na konceptu jediného Spec souboru (`.spec`). Tento soubor je deklarativním popisem celého životního cyklu balíčku:

- **%prep:** Sekce pro rozbalení zdrojového kódu a aplikaci záplat.
- **%build:** Instrukce pro kompilaci.
- **%install:** Definice instalace do virtuálního kořene (BuildRoot).
- **%files:** Explicitní seznam souborů, které má finální balíček obsahovat. Jakýkoliv soubor vytvořený v sekci %install, který není uveden v %files, způsobí chybu buildu, což zajišťuje vysokou úroveň kontroly nad obsahem balíčku.

Významným rozdílem je také práce se závislostmi. Zatímco rodina Debian využívá nástroj APT, RPM systémy přešly od YUM k modernímu nástroji DNF, který využívá pokročilé algoritmy (SAT solver) pro řešení konfliktů mezi balíčky.

1.8. Agilní metodiky a Extreme Programming

Moderní vývoj softwaru se v posledních dvou dekáдах posunul od rigidních modelů typu vodopádový model (Waterfall) k agilním přístupům, které kladou důraz na adaptabilitu a lidský faktor. Jednou z nejvlivnějších metodik v této oblasti je Extreme Programming (XP), kterou definoval Kent Beck na konci 90. let 20. století. XP se zaměřuje na zvyšování kvality softwaru a schopnost týmu reagovat na měnící se požadavky zákazníka skrze intenzivní aplikaci osvědčených inženýrských praktik [27].

Filozofie a hodnoty XP

Základem metodiky XP je pět core hodnot, které tvoří etický a profesní rámec pro veškeré rozhodování v projektu [27]:

- **Komunikace (Communication):** Většina problémů v projektech pramení z nedostatku informací. XP podporuje přímou a otevřenou komunikaci mezi členy týmu i zákazníkem.
- **Jednoduchost (Simplicity):** Cílem je vytvořit „nejjednodušší věc, která může fungovat“. Tím se minimalizuje budoucí režie a technický dluh.
- **Zpětná vazba (Feedback):** Neustálé ověřování stavu systému skrze testy a interakci se zákazníkem umožňuje včasnou korekci kurzu.

- **Odvaha (Courage):** Schopnost provádět radikální změny v návrhu (refactoring) nebo přiznat chybu a zahodit nefunkční kód.
- **Respekt (Respect):** Vzájemná úcta mezi vývojáři a důvěra v jejich schopnosti jsou nezbytné pro fungování týmových praktik.

Klíčové praktiky XP

Hodnoty jsou v XP přetaveny do konkrétních technických a procesních praktik [27]. Pro tuto bakalářskou práci byly klíčové zejména:

- **Test-Driven Development (TDD):** Psaní testů před samotným kódem zajišťuje vysoké pokrytí, definuje očekávané chování a slouží jako pojistka při refactoringu.
- **Kontinuální integrace (Continuous Integration - CI):** Časté začleňování změn do hlavní větve projektu a automatické spouštění testů pro okamžitou detekci regresí.
- **Refactoring:** Neustálé vylepšování vnitřní struktury kódu bez změny jeho vnějšího chování s cílem zachovat čistotu a udržitelnost designu.
- **Jednoduchý návrh (Simple Design):** Návrh systému odpovídá pouze aktuálním požadavkům, čímž se předejde nadměrné komplexitě (over-engineering).
- **Párové programování (Pair Programming):** Praktika, kdy dva vývojáři pracují na jednom úkolu u jednoho počítače, což zvyšuje kvalitu kódu skrze okamžité code review.

Zatímco metodika XP je primárně navržena pro týmovou spolupráci (zejména párové programování), její technické praktiky jako TDD, CI a Refactoring jsou plně aplikovatelné i pro samostatně pracujícího vývojáře a byly pro tuto bakalářskou práci klíčové.

2. Analýza a specifikace požadavků

Cílem této kapitoly je analyzovat potřeby cílové skupiny uživatelů a na jejich základě definovat funkční i nefunkční požadavky na výslednou aplikaci. Součástí je také analýza možných architektonických přístupů.

Celý proces analýzy a sběru požadavků probíhal v agilním prostředí. Během vývoje se využívaly iterativní schůzky, pravidelné standupy a dedikované brainstormové sekce. Na těchto setkáních byli přítomni zkušení vývojáři a technický ředitel (CTO), kteří měli přímý kontakt se zákazníky a znali jejich konkrétní požadavky a zpětnou vazbu. Díky tomu bylo možné rychle a efektivně reagovat na potřeby uživatelů a správně specifikovat klíčové vlastnosti aplikace.

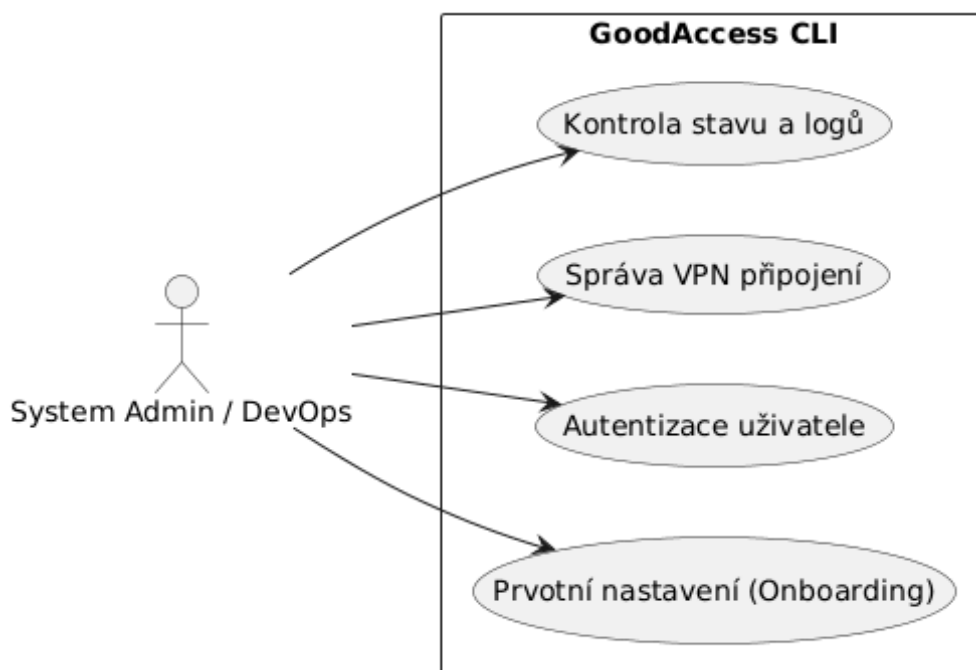
2.1. Identifikace zúčastněných stran

Primárními uživateli GoodAccess CLI pro Linux jsou:

- **System Administrators:** Spravují servery a vyžadují stabilní nástroj pro vzdálenou správu přes SSH bez nutnosti grafického rozhraní.
- **DevOps Inženýři:** Potřebují integrovat VPN připojení do automatizovaných skriptů a CI/CD pipeline. Vyžadují predikovatelné chování a strojově čitelný výstup.
- **Běžní zaměstnanci:** Uživatelé, kteří preferují textové uživatelské rozhraní (TUI) před grafickým klientem (GUI) pro každodenní práci.

2.2. Případy užití

Hlavní scénáře použití systému z pohledu koncového uživatele (administrátora či běžného zaměstnance) ukazuje následující diagram případů užití (Obrázek 2.1).



Obrázek 2.1.: Diagram případů užití GoodAccess CLI

2.3. Funkční požadavky

Na základě analýzy potřeb byly stanoveny následující funkční požadavky:

Autentizace a správa relace

Aplikace musí umožnit přihlášení uživatele v prostředí bez webového prohlížeče. Z tohoto důvodu nepodporuje přihlášení pomocí webového SSO (Single Sign-On).

- **F1 – Login:** Přihlášení probíhá pomocí standardního terminálového vstupu, kam uživatel zadá název týmu, uživatelské jméno a heslo. Zadávaní hesla je pro bezpečnost maskováno (zobrazují se pouze znaky *).
- **F2 – Logout:** Standardní ukončení relace uživatele a bezpečné smazání lokálně uložených tokenů.

Správa připojení

Jádrem aplikace je ovládání VPN tunelu a správa nastavení pro připojení.

- **F3 – Connect:** Navázání připojení. Standardně se využije preferovaný protokol a brána (gateway) uložená z předchozího nastavení.
- **F4 – Connect Flags:** Možnost dočasně přepsat preferované nastavení při připojení pomocí přepínačů `-g` (gateway) a `-p` (protocol). Přepínač `--protocol` očekává textový

název protokolu, zatímco `--gateway` vyvolá interaktivní výběr. Tato volba se neuloží jako preferovaná.

- **F5 – Disconnect:** Korektní ukončení VPN připojení a úklid routovacích pravidel.
- **F6 – Persistent Connect:** Nastavení připojení na úrovni celého zařízení, které zajistí automatické znovupřipojení i po restartu počítače.
- **F7 – Gateway Selection:** Možnost uživatele vybrat si konkrétní VPN bránu.
- **F8 – Protocol Selection:** Možnost uživatele zvolit preferovaný VPN protokol (WireGuard nebo OpenVPN).

Monitoring a stav

- **F9 – Status:** Zobrazení aktuálního stavu připojení (Připojeno/Odpojeno, IP adresa, přenesená data). Výstup lze volitelně formátovat do JSON pro snadné strojové zpracování.

Správa konfigurace a konfliktů

- **F10 – Setup:** Interaktivní průvodce prvotním nastavením (onboarding wizard). Umožňuje uživateli v jednom kroku provést přihlášení, zvolit protokol, vybrat bránu, nastavit perzistentní připojení a rovnou se připojit. Tento proces je striktně interaktivní.
- **F11 – Řešení konfliktů (Conflict Resolution):** Systém musí detekovat konfliktní stavy. Pokud je aktivní relace v grafickém klientovi (GUI) nebo je na stejném stroji připojen jiný uživatel, CLI klient na to uživatele upozorní a striktně zamezí přepsu stavu sítě.

Obecné příkazy

- **F12 – Version:** Zobrazení aktuální verze aplikace.
- **F13 – Help:** Zobrazení standardní nápovědy k dostupným příkazům a jejich použití.

2.4. Nefunkční požadavky

- **N1 – Výkon a odezva:** Aplikace v prostředí příkazové řádky (CLI) musí reagovat okamžitě. Zásadní je rychlý start aplikace a nízká paměťová náročnost, aby její spuštění uživatele neomezovalo.
- **N2 – Kompatibilita:** Aplikace musí být funkční na hlavních serverových distribucích Linuxu (Debian/Ubuntu, Fedora/RHEL, Arch Linux).

- **N3 – Bezpečnost:** Citlivá data (přístupové tokeny, privátní klíče) musí být uložena bezpečně a přístupná pouze oprávněné službě.
- **N4 – Distribuce a nasazení:** Pro usnadnění instalace a správy ze strany systémových administrátorů musí být aplikace zabalitelná do standardních distribučních balíčků. Konkrétně se jedná o formáty `.deb` (pro rodinu Debian) a `.rpm` (pro rodinu Red Hat).

2.5. Analýza architektury

Analýza stávajícího řešení GoodAccess

Aplikace GoodAccess na operačním systému Linux v současné době funguje jako rozdělený systém složený ze dvou hlavních komponent:

- **GoodAccessService (Daemon):** Tato služba běží na pozadí pod správou systému `systemd` s právy uživatele `root`. Její primární odpovědností je správa síťových rozhraní (především tunelů WireGuard), konfigurace systémového směrování (routingu) a bezpečné uchovávání kryptografických klíčů a certifikátů.
- **Grafický klient (Electron/GUI):** Uživatelské rozhraní je postaveno na frameworku Electron. Tato část běží v běžném uživatelském prostoru (bez `root` oprávnění) a funguje v podstatě jako „hloupý“ prezentační ovladač. Zobrazuje stav a přijímá příkazy od uživatele, které následně předává daemonu ke zpracování. Komunikace mezi těmito dvěma procesy probíhá lokálně, typicky pomocí Unix Domain Sockets (IPC).

Zhodnocení problému a motivace pro CLI

Tato rozdělená architektura funguje spolehlivě na běžných desktopových instalacích Linuxu. Zásadním problémem stávajícího řešení je však závislost grafického klienta na frameworku Electron. Aplikace v Electronu pro své spuštění nutně vyžaduje přítomnost zobrazovacího serveru (např. X11 nebo Wayland).

Na serverových (headless) distribucích, v prostředích pro kontinuální integraci (CI/CD) nebo uvnitř Docker kontejnerů tyto zobrazovací servery záměrně chybí. V takových případech je spuštění a ovládání grafické aplikace nemožné, přestože klíčová běhová služba (daemon) dokáže fungovat bez problému na pozadí. Z tohoto omezení vyvstává explicitní potřeba vytvořit alternativní rozhraní ve formě aplikace pro příkazovou řádku (CLI), které dokáže plnohodnotně komunikovat s existující službou a ovládat ji čistě v textovém režimu, nezávisle na grafickém prostředí.

Možné architektonické přístupy

Při návrhu řešení byly zvažovány dva přístupy k interakci s existující logikou GoodAccess:

Návrh A: Tenký klient (Wrapper)

V tomto modelu běží v systému jedna privilegovaná služba (.NET), která drží stav a vykonává operace. CLI aplikace (Go) slouží pouze jako dálkové ovládání, které posílá příkazy službě přes IPC.

Návrh B: Samostatná logika

CLI aplikace by obsahovala veškerou logiku pro připojení a správu WireGuardu a běžela by přímo pod uživatelem root, nezávisle na existující službě.

Zvolené řešení

Byl zvolen **Návrh A (Wrapper)** z následujících důvodů:

1. **Single Source of Truth:** Služba drží jednotný stav aplikace. Nedochází k desynchronizaci mezi tím, co si myslí CLI, a tím, co je reálně nastaveno v síti.
2. **Bezpečnost:** Uživatel nemusí spouštět CLI pod **rootem**. Privilegované operace provádí pouze izolovaná služba na pozadí.
3. **Znovupoužitelnost:** Využívá se již otestovaná a existující logika v .NET backendu, což je v souladu s principy XP (Simplicity).

3. Návrh řešení

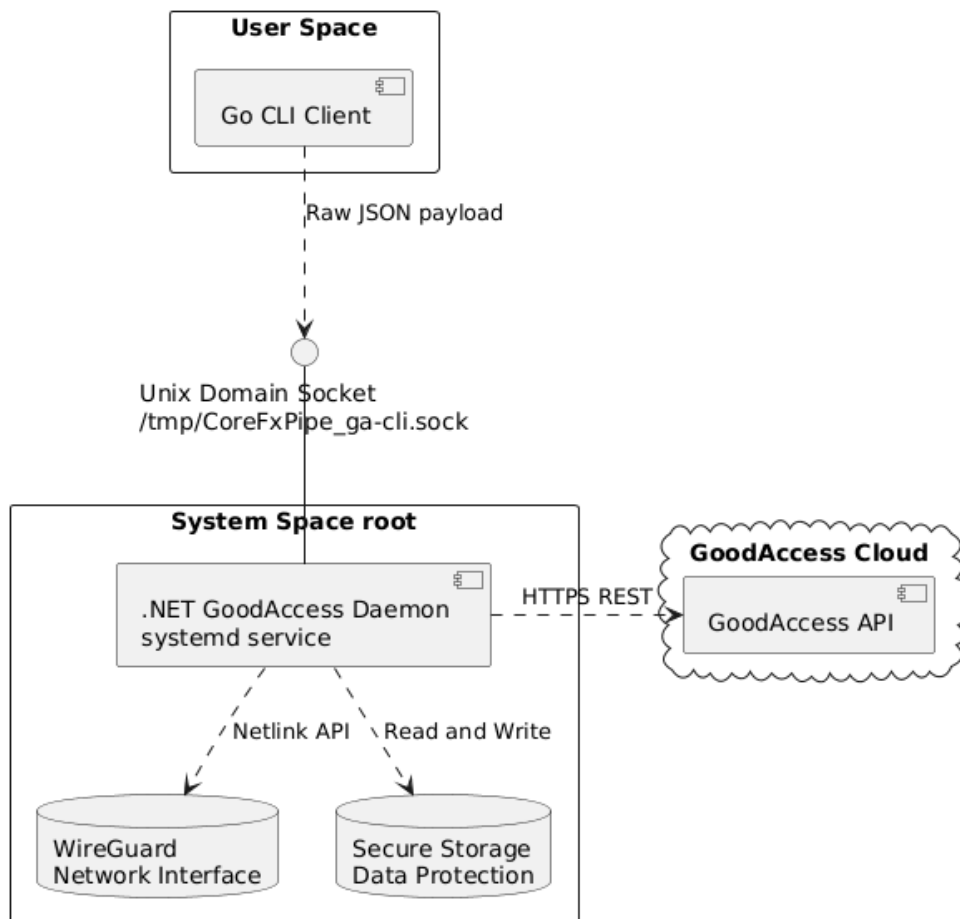
V této kapitole je představen detailní technický návrh prototypu CLI klienta a navazující síťové služby. Návrh přímo vychází ze zvolené architektury tenkého klienta, která byla zdůvodněna v předchozí kapitole (viz sekce 2.5).

3.1. Komponenty systému

Celý systém je v souladu se zvoleným návrhem rozdělen na dvě hlavní, technologicky odlišné komponenty, které spolu komunikují přes definované rozhraní.

- **Služba (Service - .NET):** Funguje jako daemon spravovaný systémem `systemd` s právy uživatele `root`. Její primární a jedinou odpovědností je správa stavu sítě, konfigurace rozhraní WireGuard a bezpečné ukládání kryptografických klíčů. Volba platformy .NET pro tuto komponentu přímo vychází z potřeby multiplatformní podpory a snadné integrace do systému `systemd` (viz sekce 1.5), a především umožňuje znovupoužití již existující a otestované logiky z grafického klienta (viz sekce 2.5).
- **Klient (Client - Go):** Běží v uživatelském prostoru (bez `root` oprávnění) a zajišťuje čistě interakci s uživatelem. Zodpovídá za parsování argumentů příkazové řádky pomocí frameworku `Cobra`, obsluhu interaktivních TUI prvků prostřednictvím `Bubble Tea` a finální formátování výstupu. Jazyk Go byl pro vývoj CLI zvolen díky svým vlastnostem pro systémové programování, rychlému startu a statické kompilaci do jediné binárky, což radikálně zjednodušuje distribuci (viz sekce 1.5 a 1.1).

Architektura Go klienta navíc úzce reflektuje principy takzvané *Clean Architecture*. Celá kódová základna klienta je rozdělena do logických vrstev, kde uživatelské rozhraní (UI a parsování příkazů) je zcela odděleno od aplikační logiky a transportní vrstvy (IPC komunikace přes Unix Domain Sockets). Toto vrstvení zaručuje vysokou testovatelnost komponent a možnost nezávisle upravovat vizuální prezentaci bez vlivu na způsob, jakým klient komunikuje s .NET službou.



Obrázek 3.1.: Architektura systému znázorňující hranici mezi .NET Službou a Go Klientem skrze IPC (Unix Domain Sockets).

3.2. Návrh komunikace IPC

Pro výměnu dat mezi Go klientem a .NET službou bylo nutné zvolit vhodný mechanismus meziprocesní komunikace (IPC). Jak bylo popsáno v teoretické části (viz sekce 1.4), pro lokální komunikaci v systému Linux jsou nejvhodnějším nástrojem Unix Domain Sockets (UDS). Ty nabízejí vyšší bezpečnost díky souborovým oprávněním a lepší výkon oproti síťovým socketům (TCP/IP), protože nedochází k režii spojené se síťovým stackem.

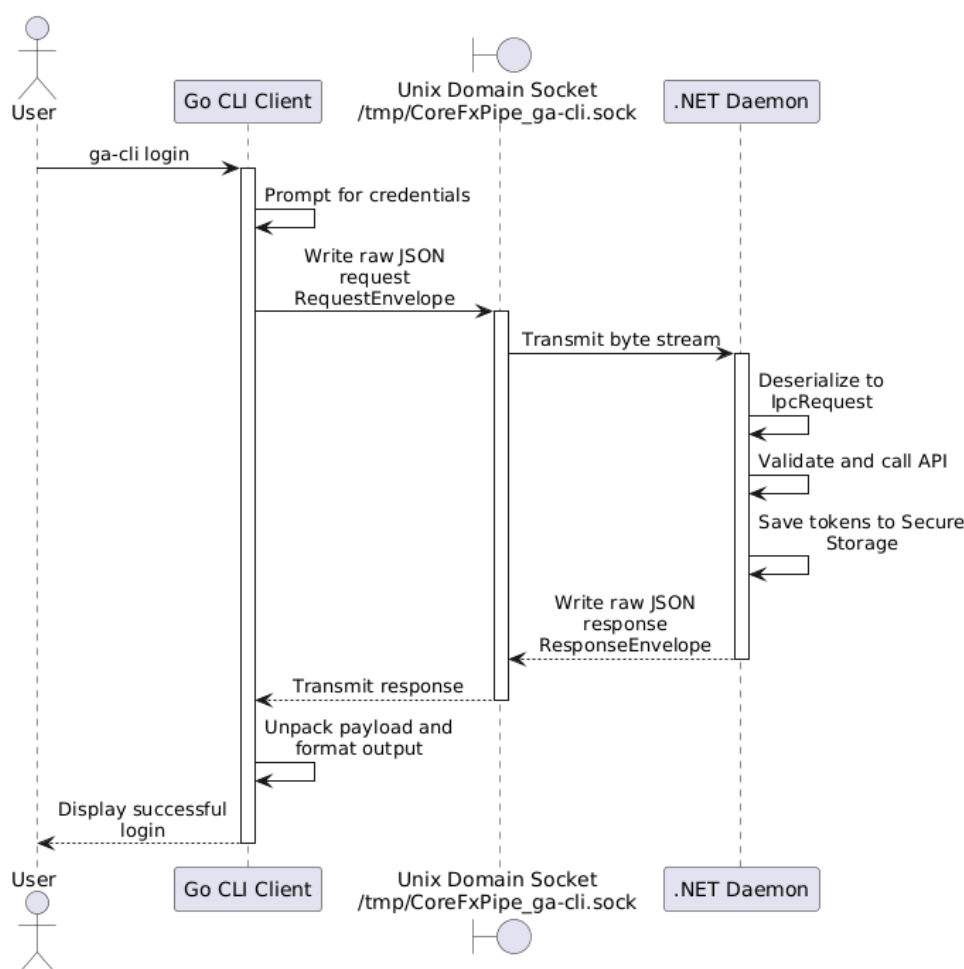
Volba formátu dat

V teoretické kapitole (viz sekce 1.5) byly porovnány formáty JSON a gRPC. Pro účely této práce byl zvolen formát **JSON**. Hlavním důvodem je jeho textová čitelnost, která výrazně usnadňuje ladění komunikace během vývoje, a nativní podpora v obou použitých jazycích (Go i C#) bez nutnosti generovat složité stuby, což odpovídá agilnímu principu jednoduchosti.

Protokol a struktura zpráv

Komunikace probíhá podle modelu Request-Response. Klient otevře spojení k socketu služby (typicky `/tmp/ga-cli.sock`), odešle požadavek v JSON formátu a čeká na odpověď, po které spojení uzavře.

Průběh typické komunikace pro přihlášení uživatele znázorňuje následující sekvenční diagram (Obrázek 3.2).



Obrázek 3.2.: Sekvenční diagram IPC komunikace při přihlášení.

Příklady zpráv

Struktura požadavku obsahuje název příkazu (**command**), samotná data (**payload**) a kontext požadavku (**context**), který nese informace o volajícím uživateli.

Příklad požadavku na přihlášení (**login**):

```
{
  "command": "login",
  "payload": {
    "TeamName": "mojefirma",
    "UserName": "jan.novak@mojefirma.cz",
    "Password": "moje_tajne_heslo"
  },
  "context": {
    "LinuxId": "1000",
    "IsRoot": false
  }
}
```

Služba na tento požadavek odpovídá strukturovanou zprávou, která indikuje úspěch či selhání a vrací příslušná data nebo chybové hlášení.

Příklad úspěšné odpovědi:

```
{
  "success": true,
  "data": {
    "IsLoggedIn": true
  }
}
```

Příklad odpovědi při chybě (neplatné údaje):

```
{
  "success": false,
  "data": null,
  "error": "Invalid credentials"
}
```

3.3. Návrh uživatelského rozhraní (CLI)

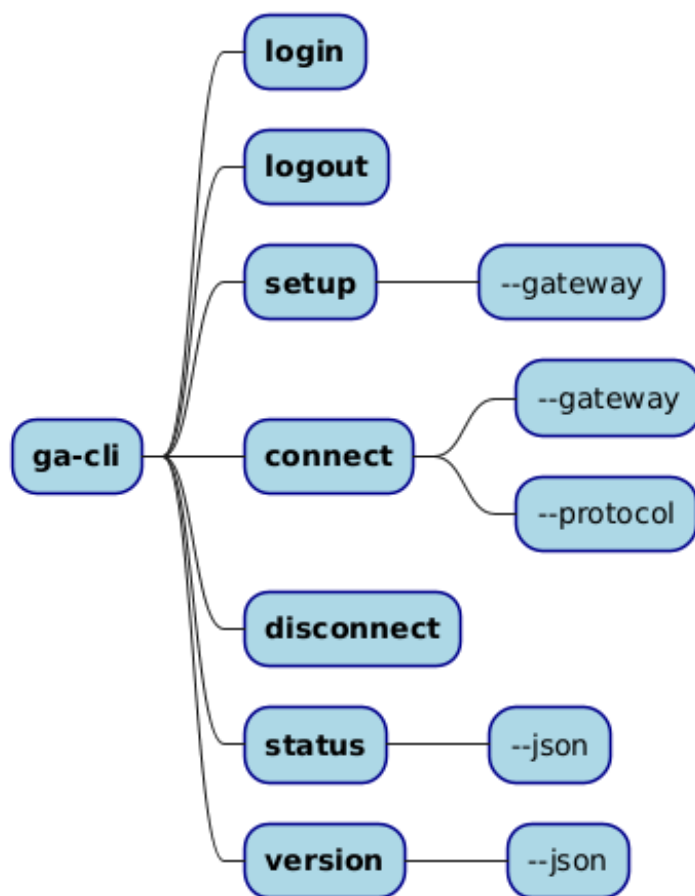
Uživatelské rozhraní aplikace je navrženo s důrazem na dvě odlišné skupiny uživatelů: lidi, kteří vyžadují interaktivní a přehledné prostředí pro každodenní práci, a stroje (skripty, CI/CD systémy), které potřebují predikovatelný a snadno parsovatelný výstup. Tento duální přístup přímo reflektuje funkční požadavky stanovené v analytické části (viz sekce 2.3).

Strom příkazů

Struktura příkazů je postavena na frameworku **Cobra** (viz sekce 1.5), který umožňuje definovat hierarchický model ovládání. Hlavní binární soubor **ga-cli** slouží jako vstupní bod,

pod kterým jsou organizovány jednotlivé funkční celky formou sub-příkazů. Tato struktura zajišťuje intuitivní ovládání a snadnou rozšiřitelnost o budoucí funkce.

Aktuální hierarchii příkazů znázorňuje Obrázek 3.3.



Obrázek 3.3.: Hierarchická struktura příkazů (Command Tree) aplikace ga-cli.

Mezi klíčové příkazy patří:

- **login**: Pro autentizaci uživatele vůči backendu GoodAccess.
- **setup**: Komplexní průvodce prvotním nastavením (viz sekce 3.3).
- **connect**: Navázání VPN připojení. Bez parametrů se připojuje k preferované bráně. Pomocí přepínače `-g` nebo `--gateway` lze vyvolat interaktivní výběr pro jednorázové připojení k jiné bráně.
- **disconnect**: Bezpečné ukončení aktivního tunelu.
- **status**: Zobrazení detailních informací o aktuální relaci.

Režimy zobrazení

Aplikace dynamicky mění své chování a způsob prezentace dat na základě kontextu, ve kterém je spuštěna.

Interaktivní režim (TUI)

V tomto režimu aplikace využívá framework **Bubble Tea** (viz sekce 1.5) k vytvoření bohatého textového rozhraní. Uživatel je navigován pomocí interaktivních prvků, jako jsou výběrové seznamy (např. při volbě brány), spinners indikující probíhající operaci nebo barevně odlišené stavové zprávy. Tento režim je aktivován automaticky, pokud je detekován standardní terminálový výstup (TTY).

Skriptovací a headless režim

Pro účely automatizace a volání z jiných programů aplikace podporuje přepínač `--json`. V tomto režimu je potlačeno veškeré interaktivní vykreslování (TUI), barvy a spinners. Výstupem příkazů je pak čistý a validní JSON objekt (viz sekce 1.5), který lze snadno zpracovat nástroji jako `jq`. Pokud aplikace detekuje, že výstup je přeměrován do roury (pipe) nebo souboru a přepínač `--json` není přítomen, automaticky vypíná barvy, aby nedocházelo ke znečištění textu ANSI únikovými kódy.

Vizuální styl a UX principy

Pro zajištění konzistentního a profesionálního vzhledu v terminálu využívá aplikace knihovnu **Lip Gloss** pro definici stylů a barevné palety. Zvolené barvy nejsou náhodné, ale plní specifickou sémantickou roli:

- **Primary (Fialová #7D56F4):** Použita pro hlavní značku aplikace, nadpisy a zvýraznění klíčových informací.
- **Secondary (Zelená #04B575):** Indikuje úspěšné operace a stav „Připojeno“.
- **Error (Červená #FF0000):** Kritické chyby a neúspěšné validace.
- **Warning (Oranžová #FFA500):** Varování, která nevyžadují okamžité ukončení, ale vyžadují pozornost uživatele.
- **Subtle (Šedá #626262):** Doplnkové informace a nápovědy, které nemají odvádět pozornost od hlavního toku dat.

Z pohledu uživatelské zkušenosti (UX) je kladen důraz na okamžitou odezvu. Každá operace trvající déle než 100 ms je doprovázena animovaným prvkem (spinner), aby uživatel věděl, že aplikace „nezamrzla“. Dále aplikace využívá kontextové nápovědy (hints), které uživateli po dokončení příkazu navrhnou další logický krok (např. po úspěšném přihlášení navrhnou spuštění `ga-cli setup` nebo `ga-cli connect`).

Průvodce nastavením (Setup)

Specifickým prvkem návrhu je příkaz `ga-cli setup`. Jedná se o tzv. „vše v jednom“ flow, které je navrženo pro maximální pohodlí při prvním spuštění aplikace. Průvodce lineárně provede uživatele následujícími kroky:

1. Autentizace (pokud uživatel není přihlášen).
2. Výběr preferovaného VPN protokolu (WireGuard / OpenVPN).
3. Interaktivní výběr výchozí brány (gateway).
4. Dotaz na aktivaci perzistentního připojení (Auto-connect).
5. Okamžité navázání spojení s nově uloženou konfigurací.

Tento přístup radikálně snižuje bariéru vstupu pro nové uživatele, kteří by jinak museli studovat manuálové stránky jednotlivých příkazů.

3.4. Správa dat a bezpečnostní model

Jelikož je navržený systém rozdělen na privilegovanou službu a neprivilegovaného klienta, je nutné zajistit bezpečné úložiště pro citlivá data (zejména autentizační tokeny) a zabezpečit meziprocessovou komunikaci. Tyto dvě oblasti spolu úzce souvisejí, a proto jsou v návrhu řešeny komplexně.

Perzistence a struktura dat

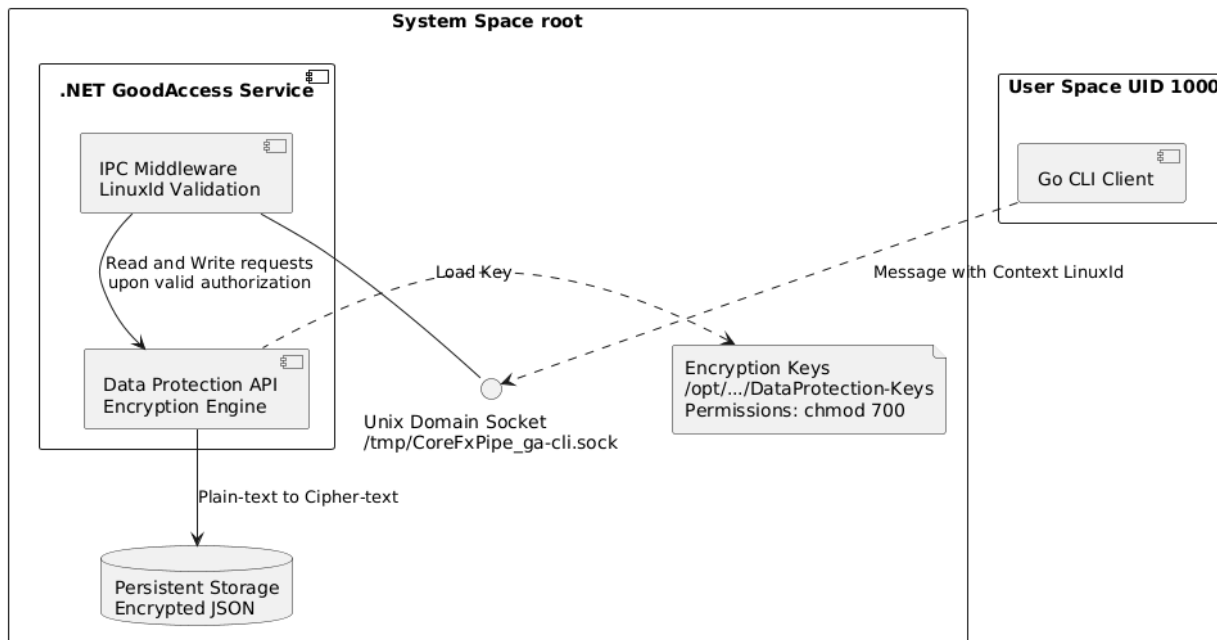
Veškerý perzistentní stav aplikace je udržován systémovou službou (daemonem), zatímco CLI klient zůstává striktně bezstavový. Datový model je definován kořenovou strukturou, která sdružuje globální konfiguraci (`GlobalConfig`), aktivní uživatelské relace obsahující relační tokeny (`ActiveSessions`) a lokální mezipaměť známých bran (`KnownGateways`). Tato datová struktura je při ukládání serializována do textového formátu JSON. Aby však nedošlo k úniku citlivých dat (například při neoprávněném čtení souborového systému jiným procesem či uživatelem), je tento textový řetězec před samotným zápisem na disk kryptograficky chráněn.

Implementace zabezpečeného úložiště (Secure Storage)

Pro ochranu perzistentních dat bylo využito rozhraní `Data Protection API`, které je nativní součástí platformy ASP.NET Core (jak bylo teoreticky popsáno v sekci 1.5). Toto rozhraní zajišťuje transparentní šifrování (`Protect`) a dešifrování (`Unprotect`) ukládaných databázových souborů.

Klíčovým bezpečnostním prvkem celého mechanismu je správa samotných šifrovacích klíčů. Ty jsou generovány a izolovány ve vyhrazeném adresáři (konkrétně `/opt/GoodAccess/configs/DataPro`).

Při startu služby je programově zajištěno nastavení striktních přístupových práv (v prostředí Linuxu pomocí příkazu `chmod 700`). Tím je na úrovni operačního systému garantováno, že k šifrovacím klíčům – a potažmo k uloženým datům – má přístup výhradně uživatel `root`, pod nímž je služba běžně spuštěna. Celková architektura bezpečnostní komponenty je znázorněna na obrázku 3.4.



Obrázek 3.4.: Architektura zabezpečeného úložiště a správy klíčů

Bezpečnost meziprosesové komunikace (IPC)

Komunikační kanál mezi klientem a službou je realizován pomocí pojmenovaných rour (Named Pipes), které jsou v prostředí operačního systému Linux implementovány jako Unix Domain Sockets (v tomto návrhu konkrétně umístěné v systémovém adresáři `/tmp/CoreFxPipe_ga-cli.sock`). Místo zavádění dedikovaných uživatelských skupin pro omezení přístupu k soketu na úrovni souborového systému je bezpečnost řešena dynamicky přímo na aplikační úrovni služby.

Každý požadavek odeslaný z CLI klienta obsahuje kromě samotných uživatelských dat (payload) také aplikační kontext s identifikátorem volajícího uživatele (`LinuxId`). Služba tak může při zpracování požadavku bezpečně ověřovat, zda má uživatel dostatečná oprávnění k požadované akci (např. manipulaci s VPN připojením, odhlášení relace). V případě zjištění narušení bezpečnosti nebo nedostatečných oprávnění je požadavek okamžitě zamítnut, čímž je zajištěna integrita systému i při sdílení jednoho soketu více lokálními uživateli.

3.5. Návrh distribuce a aktualizací

Díky cílení na platformu operačního systému Linux bylo nezbytné navrhnout distribuční a aktualizací mechanismus tak, aby ctil zavedené standardy (viz sekce 1.1) a integraci se systémovými nástroji (viz sekce 1.7). Návrh se proto cíleně vyhýbá mechanismu automatických vnitřních aktualizací (tzv. self-update), který je běžný například v prostředí Windows, a místo toho se plně spoléhá na nativní správce balíčků.

Distribuční balíčky a instalace

Aplikace bude distribuována ve dvou standardních formátech, které pokrývají drtivou většinu cílového trhu:

- **.deb balíčky** pro distribuce vycházející z rodiny Debian (např. Ubuntu, Linux Mint).
- **.rpm balíčky** pro rodinu distribucí založených na Red Hat (např. Fedora, CentOS, RHEL).

Instalační balíček funkčně zapouzdřuje obě klíčové komponenty systému. Během instalačního procesu (např. při volání `apt-get install`) dojde k umístění zkompilované binární aplikace klienta (napsané v jazyce Go) do systémového adresáře `/usr/bin/ga-cli`, čímž se stane globálně dostupnou pro všechny uživatele systému. Zároveň jsou do adresáře `/opt/GoodAccess/` rozbaleny veškeré závislosti a samotná .NET služba. Součástí instalace je také registrace příslušného konfiguračního souboru (unit file) pro inicializační systém `systemd`. Instalační skript balíčku (post-install) následně zajistí automatické spuštění služby na pozadí, čímž je systém okamžitě připraven k použití.

Detekce a provedení aktualizace

Ačkoliv je samotný proces povýšení verze (upgrade) ponechán na operačním systému, aplikace aktivně detekuje dostupnost nových vydání, aby zajistila informovanost uživatele. Tato proaktivní detekce je implementována primárně v rámci příkazu `ga-cli version` (a podobně též v příkazu `ga-cli status`). Služba při zavolání asynchronně dotazuje centrální API GoodAccess. Pokud odpověď ze serveru indikuje dostupnost novější verze (v datovém modelu reprezentováno příznakem `UpdateAvailable`), CLI klient tuto skutečnost vizuálně zdůrazní ve standardním výstupu a doporučí uživateli provést systémovou aktualizaci.

Bezpečnostní a architektonický přínos: Tento model striktně odděluje informační povinnost klienta od výkonné moci nad souborovým systémem. Neprivilegovaný uživatel je upozorněn na novou verzi, avšak samotný přepis spustitelných souborů a nezbytný restart daemonu může provést pouze systémový administrátor (`root`) standardním voláním systémového správce balíčků (např. `sudo apt-get update && sudo apt-get upgrade goodaccess-cli`). Tím je na architektonické úrovni zamezeno případnému narušení bezpečnosti (privilege escalation) skrze potenciální zranitelnosti v aktualizacím procesu samotné aplikace.

4. Implementace

V této kapitole je detailně rozebrán samotný proces implementace navrženého řešení. Pozornost je věnována struktuře celého projektu a následně specifickým technickým výzvám spojeným s vývojem klientské aplikace v jazyce Go a úpravám systémové služby na platformě .NET.

4.1. Struktura projektu a vývojové prostředí

Jelikož se výsledný produkt skládá z několika oddělených, avšak úzce spolupracujících částí, bylo zásadní správně navrhnout strukturu repozitáře. Repozitář je koncipován jako mono-repo, které zastřešuje zdrojové kódy pro všechny klientské i systémové komponenty. Tento přístup výrazně zjednodušuje správu závislostí a zajišťuje konzistentní verzování napříč celým projektem.

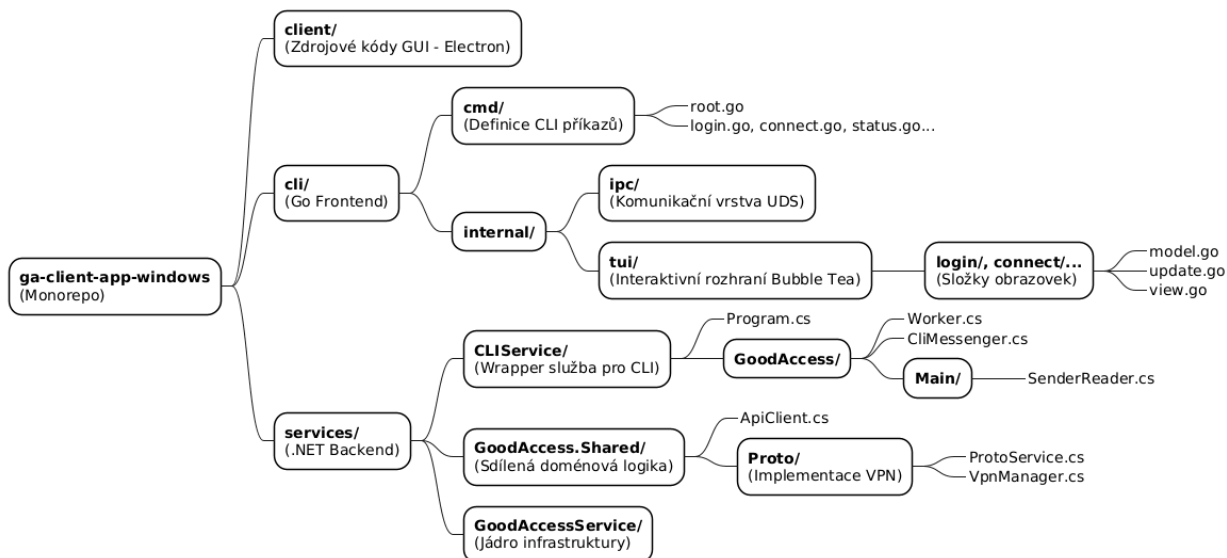
Hlavní rozdělení kořenového adresáře odráží oddělení jednotlivých technologických a logických domén:

- **client/** – Tato složka představuje zdrojové kódy původního grafického klienta postaveného na technologiích Electron a React. Byť není primárním předmětem této práce, jeho přítomnost v repozitáři usnadňuje sdílení prostředků.
- **cli/** – Zde se nachází kompletní implementace nového terminálového klienta (frontendu) v jazyce Go.
 - **cmd/** – Obsahuje definice jednotlivých CLI příkazů využívajících framework Cobra (např. `login.go`, `connect.go`, `status.go`). Soubor `root.go` slouží jako vstupní bod pro parsování uživatelských vstupů.
 - **internal/** – Zapouzdřuje interní logiku aplikace, kterou nelze importovat z externích balíčků.
 - * **ipc/** – Implementuje komunikační vrstvu, konkrétně klienta pro Unix Domain Sockets a definici zpráv (tzv. *protocol*).
 - * **tui/** – Zastřešuje interaktivní textové uživatelské rozhraní postavené na frameworku Bubble Tea. Každá obrazovka (např. `login/`, `connect/`) je zde oddělena do vlastní složky obsahující modely (`model.go`), zpracování událostí (`update.go`) a vykreslovací logiku (`view.go`).

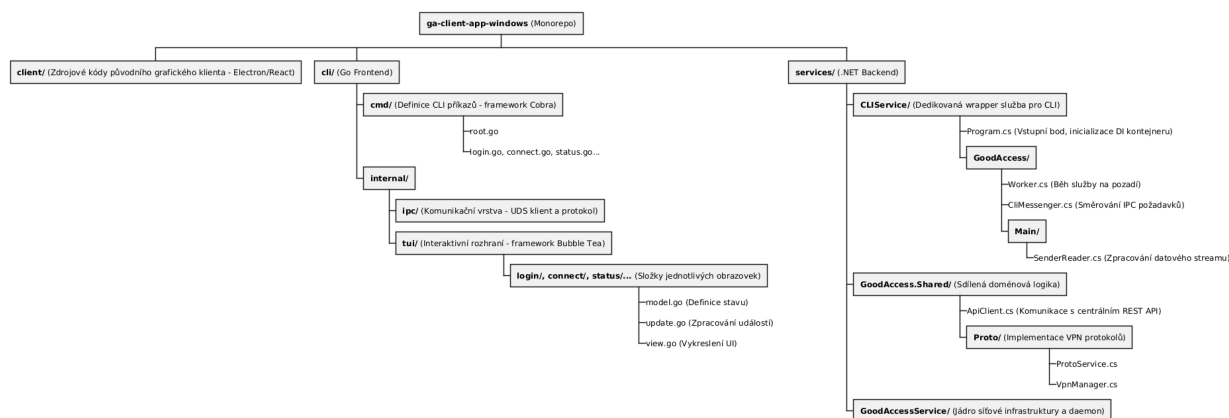
- **services/** – Obsahuje .NET projekty představující backend vrstvu, neboli systémové služby běžící na pozadí s vyššími právy.
 - **CLIService/** – Zcela nově vytvořený projekt v rámci této práce. Funguje jako dedikovaná *wrapper* služba (tzv. obálka) pro CLI klienta. Hlavním úkolem CLIService je naslouchat na IPC socketu (**SenderReader.cs**), přijímat požadavky od frontend klienta a směřovat je pomocí komponenty **CliMessenger.cs** k příslušným výkonným vrstvám.
 - **GoodAccess.Shared/** – Obsahuje sdílenou doménovou logiku, kterou využívají různé části backendu. Nachází se zde klientské třídy pro komunikaci s centrálním REST API (**ApiClient.cs**) a abstraktní implementace VPN protokolů (**ProtoService.cs**, **VpnManager.cs**).
 - **GoodAccessService/** – Původní a hlavní jádro síťové infrastruktury (daemon). Využívá konfigurace z předchozích vrstev k fyzickému sestavení a správě tunelů na daném operačním systému.

Tato architektura je tak striktně rozdělena na takzvaného tenkého klienta (Thin Client), jenž je zodpovědný výhradně za prezentaci dat a odesílání uživatelských povelů, a robustní systémovou službu, která si ponechává komplexní aplikační logiku. Navíc obě tyto logické vrstvy (Go aplikace a .NET služby) mají vlastní nezávislé procesy pro sestavení a distribuci.

Pro názornější představu o komplexnosti řešení a vzájemných vazbách jsou níže uvedeny dvě doplňující vizualizace. Obrázek 4.1 nabízí pohled na strukturu repozitáře ve formě myšlenkové mapy, zatímco Obrázek 4.2 představuje stejnou hierarchii formou WBS (Work Breakdown Structure) diagramu pro detailnější dekompozici jednotlivých submodulů.



Obrázek 4.1.: Struktura repozitáře (Mindmap)



Obrázek 4.2.: Struktura repozitáře (WBS)

4.2. Implementace systémové služby (.NET)

Aby bylo možné efektivně zpracovávat požadavky přicházející od CLI klienta, bylo nutné infrastrukturu na straně služby patřičně rozšířit. Vývoj probíhal iterativně počínaje zajištěním bezpečného úložiště a zprovozněním lokálního komunikačního kanálu.

Bezpečné úložiště (Secure Storage)

Aplikace potřebuje uchovávat citlivá data jako jsou autentizační tokeny a privátní klíče. Jak bylo definováno v kapitole 3.4, původní implementace pro grafické rozhraní ukládala některá data do prostoru uživatele. Jelikož je CLI klient navržen s možností trvalého spojení běžícího čistě na úrovni systému, muselo být úložiště přesunuto do bezpečného systémového adresáře (/opt/GoodAccess/configs/DataProtection-Keys).

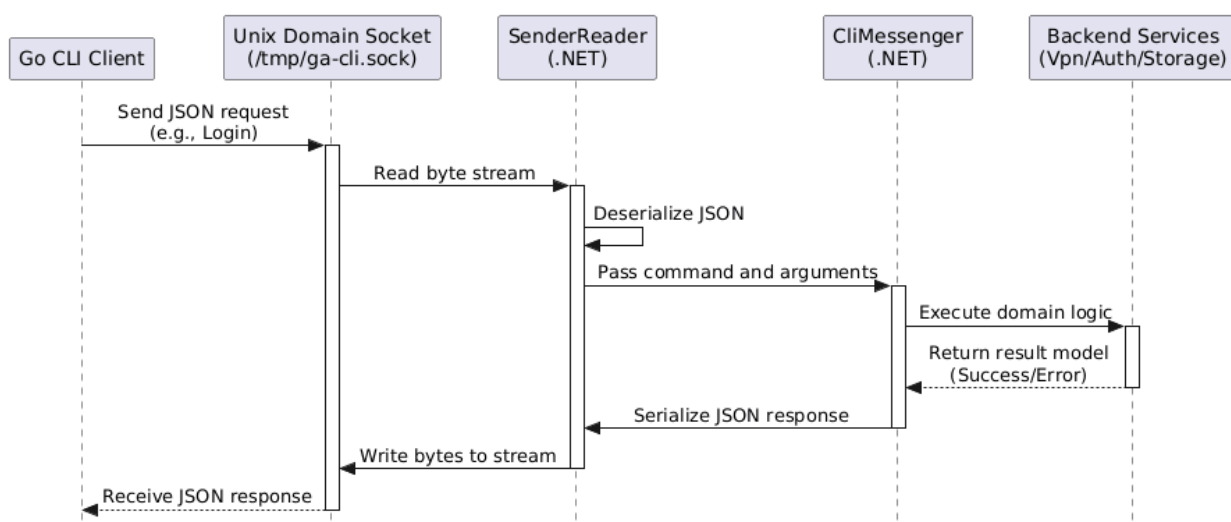
Byla vytvořena třída `CliStorage` reprezentující zabezpečené úložiště. Fyzické uložení dat využívá soubory zabezpečené restriktivními přístupovými právy, čímž je zaručeno, že k souborům má přístup pouze vlastník procesu (v tomto případě uživatel `root`, pod kterým služba běží). Samotná data uvnitř souborů jsou navíc šifrována na úrovni .NET frameworku pomocí `IDataProtectionProvider`. Třída umožňuje čtení a zápis serializovaných JSON objektů (`CliStorageRoot`) do tohoto chráněného souboru.

```
private void WriteFile(object json)
{
    using (StreamWriter sw = File.CreateText(this._path))
    {
        sw.WriteLine(this._protector.Protect(JsonSerializer.
            Serialize(json)));
    }
}
```

IPC Server (Unix Domain Sockets)

Základním kamenem architektury je nově vytvořený modul `CLIService`. Tento modul slouží jako most mezi uživatelskými vstupy a jádrem služby. Jak bylo navrženo v kapitole 3.2, komunikace probíhá přes Unix Domain Sockets (UDS), což je na platformě Linux standardní a vysoce výkonný způsob výměny dat mezi lokálními procesy. Služba vytvoří pojmenovanou rouru (named pipe), na které asynchronně naslouchá příchozím spojením od klienta. Požadavky jsou zasílány a vráceny ve formátu JSON, což usnadňuje deserializaci dat na obou stranách.

```
_serverStream = new NamedPipeServerStream(
    _pipeName,
    PipeDirection.InOut,
    1, // Max 1 client
    PipeTransmissionMode.Byte,
    PipeOptions.Asynchronous
);
```



Obrázek 4.3.: Architektura IPC komunikace

Zpracování příkazů (CliMessenger)

Za samotné odbavování příchozích zpráv zodpovídá komponenta `CliMessenger`. Jak bylo definováno v podkapitole 3.2 (včetně konkrétních JSON příkladů), každý požadavek ve formátu JSON obsahuje specifikaci příkazu (např. `login`, `connect`) a příslušné argumenty. `CliMessenger` tyto požadavky zpracovává a směřuje je na specifické služby v rámci .NET backendu. Jak ukazuje následující ukázka kódu, třída pomocí konstrukce `switch` analyzuje přijatý příkaz a asynchronně volá odpovídající obslužné metody.

```

switch (command.ToLower())
{

    case "login":
        await HandleLogin(payload, linuxId,
            responseWrapper);
        break;

    case "logout":
        await HandleLogout(linuxId, responseWrapper);
        break;
}

```

V raných fázích vývoje, kdy ještě nebyla zcela implementována veškerá produkční logika a klientská aplikace se teprve formovala, vracela metoda `CliMessenger` provizorní, pevně definované hodnoty (tzv. hardcoded data). Tento přístup umožnil paralelní vývoj – uživatelské rozhraní se mohlo designovat a testovat proti konzistentnímu API bez čekání na dokončení komplexní aplikační logiky. Postupem času byly tyto provizorní odpovědi nahrazovány reálným napojením na síťové a autentizační subsystémy.

Životní cyklus a integrace se systemd

Pro zajištění běhu služby na pozadí a správnou instalaci na Linuxu byl využit modul `Microsoft.Extensions.Hosting.Systemd`. Díky tomuto modulu se celá aplikace může chovat jako nativní `systemd` služba. Definice `goodaccess-cli.service` zajišťuje automatické spuštění po startu systému a *graceful shutdown* proces pro bezpečné odpojení VPN před vypnutím. Následující ukázka kódu z konfigurační třídy `Program.cs` demonstruje inicializaci hostitele, kde metoda `UseSystemd()` registruje životní cyklus služby a zároveň probíhá konfigurace `DataProtection` API z předchozí podkapitoly a registrace hlavní pracovní třídy `Worker`, která obstarává chod aplikace v pozadí.

```
        IHost host = Host.CreateDefaultBuilder(args)
        .UseSystemd()
        .ConfigureServices(services =>
        {
            services.AddDataProtection()
                .PersistKeysToFileSystem(new DirectoryInfo(
                    keysPath))
                .SetApplicationName("CLIService");

            services.AddSingleton(logger);
            services.AddHostedService<Worker>();
        })
        .Build();

        await host.RunAsync();
```

4.3. Implementace klientské aplikace (Go)

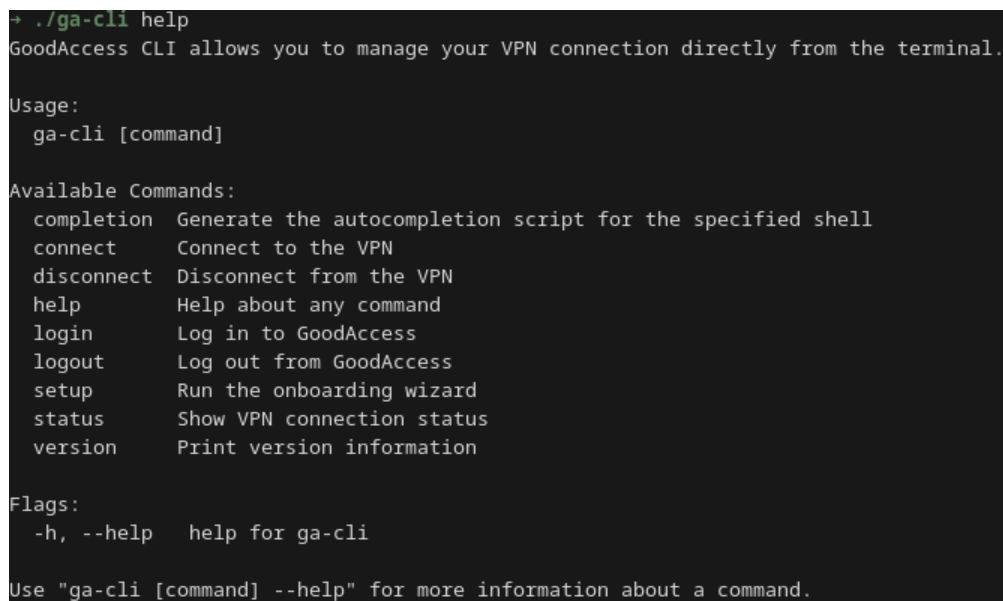
Na straně klienta bylo zvoleno využití jazyka Go, a to především pro jeho vynikající vlastnosti při vývoji CLI aplikací a schopnost kompilace do jediné statické binární formy nezávislé na externích knihovnách na koncovém systému.

Parsování příkazů (Cobra)

Základní aplikační kostra byla postavena na populárním frameworku Cobra. Tento nástroj umožňuje velmi snadno deklarovat stromovou strukturu příkazů, definovat argumenty (tzv. flags) a automaticky generovat nápovědu pro uživatele. Stromová struktura návrhu byla prezentována již v kapitole 1.5 (Obrázek 3.3). Hlavní příkaz `ga-cli` byl inicializován jako kořenový uzel (Root Command), ke kterému byly postupně přidávány dílčí příkazy odpovídající požadovaným akcím (login, setup, connect a další). Architektura frameworku Cobra navíc vybízí k přehlednému uspořádání kódu, kdy má každý příkaz svůj vlastní soubor v adresáři `cmd/`.

Následující ukázka kódu ze souboru `connect.go` ilustruje typickou definici příkazu v Cobra frameworku. Vlastnost `Use` definuje, jak se příkaz volá z terminálu. `Short` a `Long` poskytují texty pro automaticky generovanou nápovědu (Obrázek 4.4). Samotná výkonná logika je pak svěřena funkci pod klíčem `RunE`, která v tomto případě inicializuje IPC klienta a odesílá požadavek na připojení.


```
cmd := &cobra.Command{
    Use:     "connect",
    Short:   "Connect to the VPN",
    Long:    `Establishes a VPN connection to the default or specified
            gateway.`,
    RunE: func(cmd *cobra.Command, args []string) error {
        protoLower := strings.ToLower(protocol)
        if protoLower != "wireguard" && protoLower != "openvpn" {
```



```
➔ ./ga-cli help
GoodAccess CLI allows you to manage your VPN connection directly from the terminal.

Usage:
  ga-cli [command]

Available Commands:
  completion  Generate the autocompletion script for the specified shell
  connect     Connect to the VPN
  disconnect  Disconnect from the VPN
  help        Help about any command
  login       Log in to GoodAccess
  logout      Log out from GoodAccess
  setup       Run the onboarding wizard
  status      Show VPN connection status
  version     Print version information

Flags:
  -h, --help  help for ga-cli

Use "ga-cli [command] --help" for more information about a command.
```

Obrázek 4.4.: Automaticky generovaná nápověda hlavního příkazu

Interaktivní terminálové rozhraní (Bubble Tea)

Pro složitější interakce, jako je průvodce nastavením (Setup wizard), bylo využito textové uživatelské rozhraní (TUI) pomocí knihovny Bubble Tea. Jak bylo teoreticky rozebráno v kapitole 1.5, tento framework implementuje *The Elm Architecture*. Implementace využívá funkci `Update` pro reakci na klávesové vstupy a funkci `View` pro vykreslování vizuálních komponent na základě aktuálního stavu modelu. Celý onboarding proces tak působí na uživatele uceleným a přehledným dojmem.

Následující ukázka kódu z `view.go` demonstruje, jak je definován vzhled jednoho konkrétního kroku průvodce (Obrázek 4.5) – dotaz na navázání spojení. Lze si povšimnout využití balíčku `lipgloss` (zde pod aliasem `style`) pro aplikování barev a formátování textu, což výrazně zlepšuje čitelnost v terminálu.

```

case StepConnect:
    s += style.SubtleStyle.Render("Step_5:_Connect") + "\n"
    s += "Do_you_want_to_connect_to_the_VPN_now?\n\n"

    for i, opt := range m.ConnectOptions {
        cursor := ""
        st := style.SubtleStyle
        if i == m.cursor {
            cursor = ">"
            st = style.ValueStyle
        }
        s += fmt.Sprintf("%s%s\n", cursor, st.Render(opt))
    }

    s += "\n" + style.SubtleStyle.Render("[up/down]_Select_-_[Enter]_Confirm")

```

```

→ ./ga-cli setup

GoodAccess Setup

Logged in as: AndroidTest
Protocol:      OpenVPN
Gateway:      Recommended Gateway
Persistent:    Yes

Connect Now?

Do you want to connect to the VPN now?

> Yes, connect now
No, save and exit

[!/?] Select * [Enter] Confirm * [Esc] Quit

```

Obrázek 4.5.: Interaktivní krok průvodce nastavením pro navázání spojení

Komunikace se službou (IPC Klient)

Odesílání a přijímání JSON dat z Unix Domain Socketu je zajištěno IPC klientem na straně Go aplikace. Klient navazuje spojení přes systémové volání s definovaným unix socketem. Pro zajištění konzistence je každá odesílaná zpráva zabalena do tzv. *Request Envelope*. Tato obálka (implementovaná jako Go struktura) obsahuje nejen samotný povel, ale také kontextové informace, jako je ID Linuxového uživatele, který příkaz vyvolal, a flag určující, zda byl příkaz spuštěn s právy administrátora (root). Tyto údaje jsou nezbytné pro správné ověření

oprávnění na straně služby. Následně klient formátuje tyto požadavky do JSON struktur a přijímá odpovědi od .NET služby, čímž překlenuje mezeru mezi frontendem a backendem.

```
IsRoot := os.Getuid() == 0
conn, err := net.DialTimeout("unix", c.socketPath, c.timeout)
if err != nil {
    return "", fmt.Errorf("connection failed: %w", err)
}
defer func() { _ = conn.Close() }()

if err := conn.SetDeadline(time.Now().Add(c.timeout)); err != nil {
    return "", fmt.Errorf("failed to set deadline: %w", err)
}

req := ipc.RequestEnvelope{
    Command: command,
    Payload: payload,
    Context: &ipc.RequestContext{
        LinuxId: getRealUserID(),
        IsRoot: IsRoot,
    },
}
```

4.4. Pokročilé funkce a technické výzvy

Během integrace stěžejních funkčních celků do architektury se objevilo několik netriviálních technických výzev, které vyžadovaly hlubší analýzu a návrh specializovaných řešení.

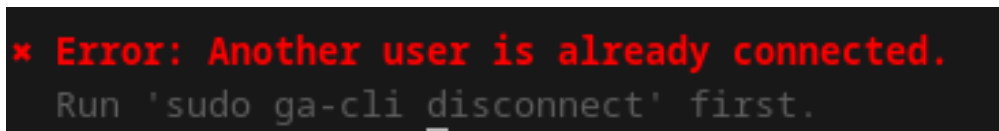
Správa VPN spojení (OpenVPN)

Samotné vytvoření šifrovaného tunelu skrze protokol OpenVPN těží z již existující architektury modulu `ProtoService`, který je nasazen na ostatních platformách. Pro potřeby CLI klienta tak stačilo mapovat požadavky z UDS rozhraní do podoby interních struktur sítě a volat sdílenou aplikační logiku. Modul automaticky vygeneruje odpovídající konfiguraci, sestaví spojení a na pozadí monitoruje stabilitu relace.

Persistentní relace a vícero uživatelů

Jednou z definovaných funkčností v rámci funkčních požadavků (kapitola 2.3) byla možnost navázání persistentního (trvalého) spojení, kdy VPN zůstává aktivní i po odhlášení aktuálního systémového uživatele nebo po restartu zařízení. Tento požadavek si vyžádal řešení problému týkajícího se izolace více Linuxových uživatelů. Jelikož je sestavené VPN spojení systémového charakteru (routování přes rozhraní tunelu ovlivňuje globální směrovací tabulky

stroje), mohlo by dojít k situaci, kdy jeden uživatel sestaví persistentní spojení a jiný uživatel po přihlášení nemá oprávnění jej modifikovat či ukončit, přestože je jeho síťový provoz rovněž šifrován. Z toho důvodu bylo nutné obohatit komunikační protokol IPC o explicitní předávání systémového kontextu a identifikaci uživatele, který požadavek vznáší. Pokud je aktivní persistentní tunel spuštěný jiným uživatelem a nejedná se o autorizovanou operaci, je akce zamítnuta na úrovni služby (Obrázek 4.6). Administrátor pak musí relaci manuálně ukončit přes příkaz vyžadující superuživatelská práva voláním `sudo ga-cli disconnect`.



Obrázek 4.6.: Chybové hlášení při pokusu o úpravu relace jiného uživatele

Implementace protokolu WireGuard

Moderní protokol WireGuard představoval v kontextu architektury významnou výzvu, jelikož práce zahrnovala implementaci samotného agenta (sestavování sítě) cíleně pouze pro prostředí Linux s využitím statických konfigurací. Na rozdíl od uceleného řízení (životní cyklus přes operační systém, dynamické doporučování gateway, automatické znovupřipojení), které přesahuje rámec této bakalářské práce a je paralelně vyvíjeno ostatními členy týmu na různých platformách, se tato konkrétní implementace zaměřuje primárně na fundamentální logiku navázání komunikace přes nativní systémové nástroje (`ip link`, `wg`). Tyto operace jsou vloženy do kontextu služeb `ProtoService` a tvoří základ budoucí robustní síťové logiky.

Vylepšení uživatelské přívětivosti (UX)

S cílem maximalizovat komfort koncových uživatelů při interakci v terminálu byl klient postupně doplňován o podpůrné funkce a vizuální prvky definované v kapitole 3.3. Byly implementovány kontextové nápovědy (tzv. *hints*), které uživateli po dokončení operace nebo při kontrole stavu navrhuje další logický krok. Například, je-li uživatel odhlášen, příkaz `status` nejen zobrazí aktuální stav, ale zároveň nabídne spuštění příkazu `ga-cli login`. Dalším důležitým UX prvkem je okamžitá vizuální odezva u déletrvajících operací (např. sestavování VPN spojení), jež byla realizována pomocí animovaných načítacích elementů (spinnerů).

Důležitým přepínačem (flag) je plná podpora pro strojově čitelné formátování výstupů prostřednictvím `--json`. Tento flag zajišťuje, že aplikace nebude vracet interaktivní textové uživatelské rozhraní ani barevně formátovaný text, ale striktně strukturovaný JSON výstup. Tato funkcionality je klíčová pro případnou integraci CLI nástroje do CI/CD řetězců (pipelines) a automatizačních skriptů, kde je vyžadováno bezobslužné řízení celého procesu připojení a zpracování jeho výsledků.

```
→ ./ga-cli status

Connection Status

Status:           Disconnected
Team:             Freetka
User:             AndroidTest
Protocol:         OpenVPN
Preferred Gateway: Recommended Gateway
Persistent:       Yes

To connect, run:
• ga-cli connect
```

Obrázek 4.7.: Zobrazení stavu s nápovědou pro další krok

```
→ ./ga-cli connect

⋮ Connecting to GoodAccess...
```

Obrázek 4.8.: Okamžitá odezva (spinner) při připojování

```
→ ./ga-cli status --json
{
  "Status": "Connected",
  "IsConnected": true,
  "Duration": "22:02:47",
  "ConnectedSince": "2026-03-12T12:33:23.5166273Z",
  "IsRecommended": true,
  "UserName": "AndroidTest",
  "TeamName": "Freetka",
  "Persistent": true,
  "IsLoggedIn": true,
  "PreferredGatewayName": "Recommended Gateway",
  "PreferredGatewayIp": "",
  "PreferredGatewayCountryCode": "XX",
  "PreferredProtocol": "OpenVPN",
  "ConnectedGatewayName": "Brana",
  "ConnectedGatewayIp": "176.102.64.172",
  "ConnectedGatewayCountryCode": "CZ",
  "ConnectedProtocol": "OpenVPN"
}
```

Obrázek 4.9.: Strojově čitelný výstup pomocí flagu --json

Rozdíly ve správě stavu: GUI vs. CLI

Jedním z nejvýznamnějších architektonických rozdílů mezi grafickým klientem (GUI) a klientem v příkazovém řádku (CLI) je způsob správy stavu a ošetření uživatelských akcí. V GUI aplikaci je běžnou praxí vizuálně omezit akce, které neodpovídají aktuálnímu stavu. Například tlačítko „Připojit“ je deaktivováno nebo skryto, dokud se uživatel úspěšně nepřihlásí. Uživatel tak nemá možnost vyvolat neplatný stav pouhým kliknutím.

Naproti tomu CLI prostředí je ze své podstaty bezstavové z pohledu samotného vstupu. Uživatel může kdykoliv napsat jakýkoliv příkaz nezávisle na aktuálním stavu aplikace na pozadí. Proto je nezbytné, aby CLI klient na každý takový příkaz reagoval předvídatelně a v případě neplatného stavu poskytl jasnou chybovou hlášku s návodem k nápravě. Například, pokusí-li se nepřihlášený uživatel o připojení k VPN (`ga-cli connect`), aplikace musí požadavek zachytit a korektně odmítnout s výzvou k přihlášení.

Tato odlišnost si vyžádala pečlivé zmapování všech možných kombinací příkazů a stavů aplikace. Následující tabulka ukazuje, jak se různé příkazy chovají v různých stavech služby.

Příkaz	Stav: Nepřihlášen	Stav: Přihlášen, odpojen	Stav: Připojen (VPN)	Stav: Jiný uživatel
<code>login</code>	Provede přihlášení.	<i>Zamítnuto</i> : Již přihlášen.	<i>Zamítnuto</i> : Již připojen.	<i>Zamítnuto</i> : Chybí práva.
<code>setup</code>	Provede přihlášení.	Spustí průvodce nastavením.	Dotaz na odpojení, poté průvodce.	<i>Zamítnuto</i> : Chybí práva.
<code>connect</code>	<i>Zamítnuto</i> : Nutno se přihlásit.	Sestaví VPN spojení.	<i>Zamítnuto</i> : Již připojen.	<i>Zamítnuto</i> : Chybí práva.
<code>disconnect</code>	<i>Zamítnuto</i> : Nutno se přihlásit.	<i>Zamítnuto</i> : Není připojeno.	Odpojí VPN.	<i>Zamítnuto</i> (mimo sudo).
<code>logout</code>	<i>Zamítnuto</i> : Není přihlášen.	Provede odhlášení.	<i>Zamítnuto</i> : Nutno se odpojit.	<i>Zamítnuto</i> : Chybí práva.
<code>status</code>	Zobrazí stav a náповědu.	Zobrazí stav a náповědu.	Zobrazí detail spojení.	<i>Zamítnuto</i> : Chybí práva.

Tabulka 4.1.: Chování CLI příkazů závislé na stavu aplikace a relace

5. Testování a validace

Ověření funkčnosti a stability aplikace probíhalo v souladu s metodikou Extrémního programování průběžně během vývoje.

5.1. Metodika testování

V souladu s principy agilního vývoje, zejména metodiky Extrémního programování (XP), bylo testování integrální součástí celého vývojového cyklu, nikoliv až jeho závěrečnou fází. Uplatňován byl přístup TDD (Test-Driven Development), kdy byly pro klíčové logické celky psány testy ještě před samotnou implementací. Tento postup nejenže zajistil vysoké pokrytí kódu, ale zároveň sloužil jako živá specifikace chování jednotlivých komponent [27]. Metodika TDD v XP klade důraz na krátké vývojové cykly, které začínají napsáním selhávajícího testu (fáze *Red*), následuje implementace minimálního množství kódu pro jeho splnění (fáze *Green*) a proces je zakončen refaktoringem kódu při zachování funkčnosti testů. Tento iterativní proces umožňuje udržet vysokou kvalitu kódu, minimalizovat technický dluh a poskytuje vývojáři okamžitou zpětnou vazbu o správnosti řešení.

Strategie testování vychází z konceptu testovací pyramidy, který byl detailně popsán v teoretické části práce (viz podkapitola 1.6). Tato hierarchie zajišťuje, že chyby jsou detekovány v co nejranější fázi vývoje, kdy je jejich oprava nejméně nákladná. V rámci této práce byly realizovány testy na všech úrovních pyramidy, od rychlých jednotkových testů až po komplexní akceptační scénáře.

5.2. Jednotkové testy a mockování

Největší důraz byl kladen na jednotkové testy, které umožňují rychlou a spolehlivou verifikaci aplikační logiky.

Testování klienta v Go

Pro testování CLI klienta napsaného v jazyce Go byl využit standardní vestavěný balíček `testing` v kombinaci s knihovnou `testify` (teoretický rozbor viz podkapitola 1.5). Tato kombinace poskytuje bohatší sadu nástrojů pro aserce a především pro mockování závislostí.

Klíčovou technikou bylo mockování pro rozhraní meziprocessové komunikace (IPC). Jelikož frontendová aplikace v Go komunikuje se backendovou službou v .NET přes Unix Domain Socket, bylo pro účely testování nezbytné tuto závislost odstranit. Tímto způsobem bylo možné testovat logiku a vzhled textového uživatelského rozhraní (TUI) zcela izolovaně, bez nutnosti mít spuštěnou a nakonfigurovanou systémovou službu.

Byl vytvořen mock klient `MockClient`, který implementuje stejné rozhraní jako reálný IPC klient. Pomocí knihovny `testify/mock` bylo možné předdefinovat očekávané chování – tedy jakou odpověď má mock vrátit, když je zavolána jeho metoda `Send` s určitými parametry.

Listing 5.1: Ukázka mock klienta pro IPC komunikaci

```
package ipc

import "github.com/stretchr/testify/mock"

type MockClient struct {
    mock.Mock
}

func (m *MockClient) Send(command string, payload any) (string, error) {
    {
        args := m.Called(command, payload)
        return args.String(0), args.Error(1)
    }
}

func (m *MockClient) Close() error {
    {
        args := m.Called()
        return args.Error(0)
    }
}
```

Tento `MockClient` byl následně v testovacích scénářích (např. v souboru `setup_test.go`) injektován do komponenty TUI namísto reálného klienta. Test tak mohl snadno simulovat různé scénáře, jako je úspěšné přihlášení, chyba spojení nebo příjem dat ze služby, a ověřit, že se uživatelské rozhraní chová dle očekávání.

Testování služby v .NET

Na straně systémové služby v .NET byly jednotkové testy implementovány s využitím frameworku `xUnit` (viz podkapitola 1.5). Návrh počítal s testováním jednotlivých tříd v izolaci, přičemž pro odstranění závislostí byl využit framework `Moq`. Příkladem může být testování třídy `CliStorage`, která zodpovídá za bezpečné ukládání dat.

Následující ukázka kódu demonstruje testovací scénář pro třídu `CliStorage`, kde je pomocí mockování ověřeno, že jsou data před uložení správně šifrována.

Listing 5.2: Ukázka jednotkového testu v .NET pomocí xUnit a Moq

```

using Xunit;
using Moq;
using GoodAccess.CLI.Storage;

namespace GoodAccess.CLI.Tests
{
    public class CliStorageTests
    {
        [Fact]
        public void SavePreferences_ShouldStoreCorrectData()
        {
            // Arrange
            var mockProtector = new Mock<IDataProtectionProvider>();
            var storage = new CliStorage(mockProtector.Object);
            var prefs = new CliPreferences { LastUser = "test@example.com" };

            // Act
            storage.SavePreferences(prefs);

            // Assert
            // Verify that data was passed to the protection provider
            // and then stored
            mockProtector.Verify(p => p.Protect(It.IsAny<byte []>()),
                                Times.Once);
        }
    }
}

```

Další testovací scénáře pro třídu `CliStorage` zahrnují:

- `SavePreferences_ShouldStoreCorrectData()`: Test ověří, že po zavolání metody `SavePreferences` je do souboru zapsán správně naformátovaný a zašifrovaný JSON objekt.
- `GetRefreshToken_ShouldThrowOnMissingToken()`: Test zajistí, že pokud se metoda pokusí přechíst neexistující token, je vyhozena očekávaná výjimka.
- `SetRefreshToken_ShouldUpdateSession()`: Ověří, že zavolání metody pro nastavení nového obnovovacího tokenu správně aktualizuje data v úložišti.

Tento přístup, založený na striktním oddělení závislostí a jejich mockování, je základním předpokladem pro udržitelný a spolehlivý vývoj komplexních systémových nástrojů. Tato implementace navazuje na principy popsané v kapitole 4, kde byla zdůrazněna modularita

a bezpečnost.

5.3. Akceptační testování

Pro ověření, že se aplikace chová v souladu s očekáváním uživatele, byla využita metodika BDD (Behavior-Driven Development). Specifikace chování byly psány v jazyce Gherkin, který je snadno čitelný i pro netechnické členy týmu. Tyto specifikace, uložené v souborech s příponou `.feature`, sloužily jako formální akceptační kritéria, která byla manuálně ověřována před každým vydáním.

Tento přístup zajistil, že vývoj byl neustále v souladu s požadavky a že finální produkt odpovídá zadaným cílům. Příkladem takové specifikace je scénář pro přihlášení uživatele, uvedený v souboru `login.feature`.

Listing 5.3: Ukázka akceptačního testu pro přihlášení v jazyce Gherkin

```
Feature: Login
  As a user
  I want to log in to the GoodAccess CLI
  So that I can access the VPN network

  Background:
    Given the GoodAccess service is running

  Scenario: Successful login
    Given I am not logged in
    When I run "ga-cli login"
    And I enter a valid Team name
    And I enter a valid Username
    And I enter a valid Password
    And I submit the form
    Then the system should authenticate the user
    And the application should exit with success
```

Jednotlivé scénáře využívají klíčová slova Gherkinu, jejichž sémantika byla popsána v teoretické části práce (viz podkapitola 1.6). Tento formát přesně a jednoznačně popisuje interakci uživatele se systémem a očekávaný výsledek.

5.4. Systémové a integrační testování v prostředí Linux

Kromě jednotkových a akceptačních testů bylo klíčové ověřit funkčnost celé aplikace v reálném prostředí a potvrdit splnění nefunkčních požadavků na kompatibilitu (viz podkapitola

2.4). Testování probíhalo na třech hlavních distribucích Linuxu, které pokrývají nejrozšířenější balíčkovací systémy:

- **Fedora** jako primární vývojové prostředí.
- **Ubuntu (VM)** reprezentující rodinu Debian.
- **Arch Linux** pro ověření funkčnosti na distribucích typu rolling-release.

Cílem systémového testu bylo ověřit celý životní cyklus aplikace z pohledu uživatele. Pro zajištění objektivitu a nezávislosti nebyla tato fáze prováděna autorem práce, ale byla verifikována profesionálním QA (Quality Assurance) testerem. Proces zahrnoval:

1. **Instalace:** Úspěšná instalace balíčku (`.rpm` pro Fedoru, `.deb` pro Ubuntu, manuální kompilace/AUR pro Arch Linux).
2. **Oprávnění:** Kontrola, že hlavní spustitelný soubor `ga-cli` má po instalaci správně nastavená oprávnění (očekáváno 700), aby se zamezilo neoprávněnému spuštění jinými uživateli.
3. **Připojení:** Spuštění příkazu `ga-cli connect` a úspěšné navázání VPN spojení.
4. **Validace konektivity:** Po úspěšném připojení bylo provedeno ověření funkčnosti tunelu pomocí příkazu `ping` na interní firemní zdroj, který není dostupný z veřejného internetu.

Tento proces testování na různých distribucích pod dohledem QA testera zajistil, že aplikace je nejen funkční, ale i správně nakonfigurovaná a bezpečná pro nasazení v cílovém prostředí.

Závěr

Cílem této bakalářské práce byl návrh a implementace nativního CLI klienta GoodAccess pro OS Linux.

Shrnutí výsledků

Diskuse a přínos

Možnosti budoucího rozvoje

Seznam použitých zdrojů

1. RAYMOND, Eric Steven. *The Art of Unix Programming*. Addison-Wesley Professional, 2003. ISBN 978-0-13-142901-7.
2. WARD, Brian. *How Linux Works: What Every Superuser Should Know*. 3rd. No Starch Press, 2021. ISBN 978-1-7185-0040-2.
3. CHARMBRACELET. *Bubble Tea: A powerful little TUI framework* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://github.com/charmbracelet/bubbletea>.
4. GOODACCESS. *Zero Trust Network Access Explained* [online]. 2023. [cit. 2026-03-09]. Dostupné z: <https://www.goodaccess.com/zero-trust-network-access-ztna>.
5. DONENFELD, Jason A. WireGuard: Next Generation Kernel Network Tunnel. In: *NDSS 2017: Network and Distributed System Security Symposium*. 2017. Dostupné také z: <https://www.wireguard.com/papers/wireguard.pdf>.
6. CLOUDFLARE. *Castle-and-moat network security* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://www.cloudflare.com/learning/access-management/castle-and-moat-network-security/>.
7. NIST. *Zero Trust Architecture (SP 800-207)* [online]. 2020. [cit. 2026-03-02]. Dostupné z: <https://csrc.nist.gov/publications/detail/sp/800-207/final>.
8. CLOUDFLARE. *What is Zero Trust?* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://www.cloudflare.com/learning/security/glossary/what-is-zero-trust/>.
9. GOODACCESS. *2. Architecture Overview* [online]. 2024. [cit. 2026-02-12]. Dostupné z: <https://support.goodaccess.com/getting-started/2.-architecture-overview>.
10. PRESS, GSC Online. *Zero Trust Network Access: A Review* [online]. 2025. [cit. 2026-03-02]. Dostupné z: <https://gsconlinepress.com/journals/gscarr/sites/default/files/GSCARR-2025-0036.pdf>.
11. PRICE, Mark J. *C# 12 and .NET 8 – Modern Cross-Platform Development Fundamentals*. 8th. Packt Publishing, 2023. ISBN 978-1-83763-587-0.
12. MICROSOFT. *.NET Generic Host* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/core/extensions/generic-host>.

13. MICROSOFT. *ASP.NET Core Data Protection Overview* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/introduction>.
14. GO AUTHORS. *The Go Programming Language Specification* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://go.dev/ref/spec>.
15. DONOVAN, Alan A. A.; KERNIGHAN, Brian W. *The Go Programming Language*. Addison-Wesley Professional, 2015. ISBN 978-0-13-419044-0.
16. COBRA AUTHORS. *Philosophy of Cobra* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://cobra.dev/docs/explanations/philosophy/>.
17. HASKELL, Andy. *Intro to Bubble Tea in Go* [online]. 2022. [cit. 2026-03-02]. Dostupné z: <https://dev.to/andyhaskell/intro-to-bubble-tea-in-go-21lg>.
18. AUTHORS, The Go. *testing - The Go Programming Language* [online]. 2026. [cit. 2026-03-17]. Dostupné z: <https://pkg.go.dev/testing>.
19. STRETCHR. *Stretchr/Testify* [online]. 2026. [cit. 2026-03-17]. Dostupné z: <https://github.com/stretchr/testify>.
20. CONTRIBUTORS, xUnit.net. *xUnit.net* [online]. 2026. [cit. 2026-03-17]. Dostupné z: <https://xunit.net/>.
21. BRAY, T. *The JavaScript Object Notation (JSON) Data Interchange Format (RFC 8259)* [online]. 2017. [cit. 2026-03-02]. Dostupné z: <https://datatracker.ietf.org/doc/html/rfc8259>.
22. IBM CLOUD EDUCATION. *gRPC vs. REST: What's the Difference?* [online]. 2024. [cit. 2026-03-02]. Dostupné z: <https://www.ibm.com/cloud/blog/grpc-vs-rest>.
23. COHN, Mike. *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, 2009. ISBN 978-0-321-57936-2.
24. LUMINOUSMEN. *Understanding Upstream and Downstream in Data Pipelines* [online]. 2023. [cit. 2026-03-02]. Dostupné z: <https://luminousmen.medium.com/data-engineering-terminology-understanding-upstream-and-downstream-in-data-pipelines-bd00381515b5>.
25. DEBIAN PROJECT. *Debian Policy Manual* [online]. 2024. [cit. 2024-11-07]. Dostupné z: <https://www.debian.org/doc/debian-policy/>.
26. FEDORA PROJECT. *Fedora Packaging Guidelines* [online]. 2024. [cit. 2026-02-18]. Dostupné z: <https://docs.fedoraproject.org/en-US/packaging-guidelines/>.
27. BECK, Kent; ANDRES, Cynthia. *Extreme Programming Explained: Embrace Change*. 2nd. Addison-Wesley, 2004. ISBN 978-0-321-27865-4.

A. Externí přílohy

Externí přílohy této bakalářské práce jsou umístěny na adrese:

https://github.com/Jiri-Fiser/thesis_ki_ujep.

Na úložišti GitHub mohou být uloženy tyto externí přílohy:

- **zdrojové kódy**
- **doplňkové texty** (například jak instalovat aplikaci, manuály aplikace)
- **schémata** (především, pokud se nevejdou na stranu A4 a jejich vytištění je tak problematické)
- **screenshoty** (v textu práce lze použít jen omezený počet snímků obrazovky, které navíc nemusí být při černobílém tisku příliš přehledné)
- **videa** (například ovládání aplikace)

V každém případě by to však měli být pouze materiály, které jste vytvořili sami. Materiály jiných autorů uvádějte v seznamu použité literatury (včetně případných odkazů na jejich originální umístění).

V této kapitole stačí uvést pouze základní strukturu úložiště (co se kde nalézá a jakou má funkci) například v podobě tabulky.

ki-thesis.pdf	text práce v PDF
ki-thesis.tex	zdrojový kód práce v L ^A T _E Xu
kitheses.cls	definice třídy dokumentů (rozšířená třída scrbook)
thesis.bib	bibliografická databáze (exportována z citace.com)
LOGO_PRF_CZ_RGB_standard.jpg	logo fakulty s českým textem
LOGO_PRF_EM_RGB_standard.jpg	logo fakulty s anglickým textem

Všechny tyto soubory jsou potřeba pro překlad dokumentu (logo stačí jedno v příslušné jazykové verzi).

B. Další přílohy

Výjimečně může práce obsahovat i další tištěné přílohy. Obecně však dávejte přednost elektronickým přílohám umístěným na GitHubu (tato kapitola tak bude úplně chybět).